

**PROJECT REPORT
ON
STEGANOGRAPHY**
(A Technique to Hide Information within Image File)

**SUBMITTED IN PARTIAL FULFILLMENT FOR AWARD OF DEGREE
IN
COMPUTER SCIENCE AND ENGINEERING
(BATCH 2018-2019)**

Authors & Developers

Rahul Chhinparia

Vipul Yadav

Sachin Rana

**BACHELOR OF TECHNOLOGY
COMPUTER SCIENCE ENGINEERING**



**DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY,
LUCKNOW**

MAY, 2019

DECLARATION

We hereby declare that the entire work embodied in the Project entitled “**A Technique to Hide Information within Image File**” has been carried out by us under the supervision and guidance of **Mr. Pushpendra Kumar**. This submission is our work and that, to the best of our knowledge and belief, it contains no material previously published or written by another nor material which to a substantial extent has been accepted for the award of any Degree or Diploma to University or any other University/Institution.

ABSTRACT

Steganography is the art of hiding the fact that communication is taking place, by hiding information in other information. Many different carrier file formats can be used, but digital images are the most popular because of their frequency on the internet. For hiding secret information in images, there exists a large variety of steganography techniques some are more complex than others and all of them have respective strong and weak points. Different applications may require absolute invisibility of the secret information, while others require a large secret message to be hidden. This project report intends to give an overview of image steganography, its uses and techniques. It also attempts to identify the requirements of a good steganography algorithm and briefly reflects on which steganographic techniques are more suitable for which applications

Keywords: Encryption, Decryption, Public Key Steganography, Digital Hiding Tool.

Table of Contents

1.	INTRODUCTION	1
	1.1 Definition	2
	1.1.1 How to Use Steganography	3
	1.1.2 Statistical Preservation	3
	1.1.2.1 Embedding by Pixel Swapping	4
	1.1.2.2 New Statistical Restoration Scheme	6
	1.1.2.3 Security Analysis	7
	1.2 Problem Statement	10
	1.3 Project Overview	10
	1.4 Project Objective	13
	1.5 Scope	13
	1.6 History of Steganography	13
	1.7 Definition, Acronyms and Abbreviations	14
	1.8 System Analysis	14
2.	METHODOLOGY	16
	2.1 Detecting Steganography	16
	2.2 Stego Tools	16
	2.3 Steganography vs Cryptography	17
	2.4 Steganography vs Watermarking	17
	2.5 Steganography Techniques	18
	2.5.1 LSB	18
	2.5.2 SIHS	19
	2.5.3 Image Steganography and Bitmap Pictures	19
	2.6 Specific Requirements	21
	2.6.1 External Interface requirements	21
	2.6.1.1 Software Interface	21
	2.6.1.2 Hardware Requirements	21
	2.6.2 Functional Requirements	21
	2.6.3 Other requirements	22
3.	ANALYSIS MODELES	23
	3.1 Stego Library Class	23

4.	INSTRUCTIONS & USER INTERFACE	26
	4.1 User Manual	26
	4.2 Encryption	27
	4.3 Decryption	33
5.	SYSTEM TESTING	36
	5.1 Introduction	36
	5.2 Methods used	36
	5.2.1 Bitmap Steganography	36
	5.2.2 Encryption Process	37
	5.2.3 Decryption Process	38
	5.2.4 Sample Code	39
6.	CONCLUSION	50
7.	FUTURE SCOPE	51
8.	BIBLIOGRAPHY	52
9.	APPENDIX	53
	8.1 Introduction to Operational Steps and Methods	53
	8.2 Introduction to Theories used to Adapt Particular Design	53

LIST OF FIGURES

Fig.1.1.2.1 PSNR observed and Pixel Swapping	6
Fig.1.1.2 Performance Comparison of the proposed restoration scheme	8
Fig.1.3 The Model for Steganography	11
Fig.1.8 The Graphical Representation of the System	15
Fig.2.4 Delectability of a vessel with an introduced data	17
Fig.2.5.3 Image Steganography and Bitmap Pictures Differences	20
Fig.3.2 Hiding Information into the Image Using Encryption Module	24
Fig.3.3 Get the Hidden Information in an Image File by Decryption Process	25

Chapter 1

INTRODUCTION

One of the reasons that intruders can be successful is the most of the information they acquire from a system is in a form that they can read and comprehend. Intruders may reveal the information to others, modify it to misrepresent an individual or organization, or use it to launch an attack. One solution to this problem is, through the use of steganography. Steganography is a technique of hiding information in digital media. In contrast to cryptography, it is not to keep others from knowing the hidden information but it is to keep others from thinking that the information even exists.

Steganography become more important as more people join the cyberspace revolution. Steganography is the art of concealing information in ways that prevents the detection of hidden messages. Steganography include an array of secret communication methods that hide the message from being seen or discovered.

Due to advances in ICT, most of information is kept electronically. Consequently, the security of information has become a fundamental issue. Besides cryptography, steganography can be employed to secure information. In cryptography, the message or encrypted message is embedded in a digital host before passing it through the network, thus the existence of the message is unknown. Besides hiding data for confidentiality, this approach of information hiding can be extended to copyright protection for digital media: audio, video and images.

The growing possibilities of modern communications need the special means of security especially on computer network. The network security is becoming more important as the number of data being exchanged on the internet increases. Therefore, the confidentiality and data integrity are requiring to protect against unauthorized access and use. This has resulted in an explosive growth of the field of information hiding

Information hiding is an emerging research area, which encompasses applications such as copyright protection for digital media, watermarking, fingerprinting, and steganography.

In watermarking applications, the message contains information such as owner identification and a digital time stamp, which usually applied for copyright protection.

Fingerprint, the owner of the data set embeds a serial number that uniquely identifies the user of the data set. This adds to copyright information to make it possible to trace any unauthorized use of the data set back to the user.

Steganography hides the secret message within the host data set and is imperceptible and is to be reliably communicated to a receiver. The host data set is purposely corrupted, but in a covert way, designed to be invisible to an information analysis.

Why This Steganography?

This technique is chosen, because this system includes not only imperceptibility but also undetectability by any steganalysis tool.

Encryption transforms message to noise-like data, so the data confidentiality is protected. But the attacker knows that there is a communication, even though the data is noise-like, between sender and receiver. Steganography/data hiding/watermarking, all of them, more or less try to conceal the fact that you are involved in a secret communication.

However, for steganography/data hiding/watermarking, there are quite few works studying the computational hardness of such systems. From my personal point of view, the reasons are basically: a) not like encryption provides confidentiality (a single goal), steganography/data hiding/watermarking aims to solve a lot of questions (many goals), so it is hard to define a general adversary game for all goals (but for encryption the adversary game vs confidentiality is well defined so it can be proved by reduction finally); b) most steganography/data hiding/watermarking are actually composed systems, they themselves are relied on cryptographic tools, this further complicates the model. So, if any steganography/data hiding/watermarking can be proved to be computationally secure (for the specific goal), its real adoption could be much convincing.

1.1 Definition:

It is the art of hiding information imperceptibly in a cover medium. The word "*Steganography*" is of Greek origin and means "*covered or hidden writing*". The main aim in steganography is to hide the very existence of the message in the cover medium.

1.1.1 How to use Steganography & Tools?

Steganography

Steganography and cryptography are counter parts in digital security the obvious advantage of steganography over cryptography is that messages do not attract attention to themselves, to messengers, or to recipients. Also, the last decade has seen an exponential growth in the use of multimedia data over the Internet. These include Digital Images, Audio and Video files. This rise of digital content on the internet has further accelerated the research effort devoted to steganography. The various applications of steganography include secure military communications, multimedia watermarking and fingerprinting applications for authentication purposed to curb the problem of digital piracy. Although these are not perfect applications of steganography, many steganographic algorithms can be employed for these purposes as well.

In this thesis we have explored two different approaches to steganography. The first approach as explained in next section is based on statistical preservation which preserves the statistics of an image which get distorted due to embedding and can be exploited by targeted attacks. The second approach is based on preventing calibration based blind attacks which try to classify an image as stego or cover by trying to estimate the cover image statistics from the stego image by a process termed as Self

1.1.2 Statistical Preservation

Statistical undetectability is one of the main aspects of a steganographic algorithm. To maintain statistical undetectability, the steganographic techniques are designed with the aim of minimizing the artifacts introduced in the cover signal by the embedding technique. The main emphasis is on minimizing the noise added by embedding while increasing the payload. This is an important consideration in the design embedding algorithms, since the noise added effects the statistical properties of a medium. For a given medium, the steganographic algorithm which makes fewer

From the point of view of the steganalyst, the steganalytic attacks try to examine a signal and look for statistics which get distorted due to embedding. These statistics range from marginal statistics of first and second order in case of targeted attacks to extremely high order statistics (up to 9th order) in the case of blind steganalytic techniques which use machine learning techniques for estimating a model of the cover image from these high order statistics and reports an image to be

containing steganographic embedding if it does not conform to this model. *So, in order to defeat the steganalytic attacks, there has been a shift from the above mentioned data hiding paradigm. Algorithms have been proposed which try to restore the statistics which get distorted during the embedding procedure and which may be used for steganalysis.*

In recent literature few algorithms have been proposed where marginal statistics are preserved for achieving more security. Solanki et al have proposed an algorithm where they have compensated the first order statistics (Image histogram) after the steganographic embedding. The compensation algorithm is based on a theorem proved by Tschoepe which tries to convert one vector x into another vector y while satisfying a Minimum Mean Square Error (MMSE) criterion. This algorithm suffers from the following shortcomings:

- The algorithm assumes the cover image to be a Gaussian cover. The algorithm does not give good results for non-Gaussian cover images.
- The algorithm has been tried specifically for Quantization Index Modulation algorithm the algorithm is not giving good results for some well-known steganographic embedding such as Least Significant Bit (LSB) Replacement, LSB matching etc.
- They have ignored low probability image regions for embedding due to erratic behavior in low probability tail

In this work we propose a simple but effective embedding scheme called “Pixel Swapping” which inherently preserves the first order statistics of the cover image. Along with that a new scheme for statistical restoration has been proposed which overcomes the shortcomings of the scheme.

The details of the proposed schemes are given in the next subsection.

1.1.2.1 Embedding by Pixel Swapping:

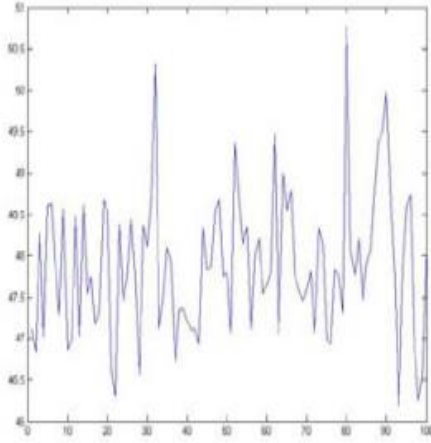
In order to maintain the first order statistics of the cover image, we have proposed a scheme which inherently preserves the first order properties of the signal. The scheme is outlined below: -

Algorithm Pixel Swap

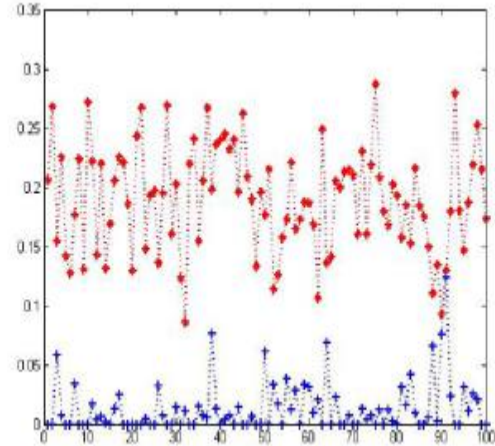
- 0 Randomly select 2 pixels x_1 and x_2 from the cover image using a pseudo –random sequence.
 - 0 If the two pixels lie within a specified distance α ($\alpha=2$ or 3 generally), they are suitable for embedding, otherwise generate another set of pixels.
 - 0 Pick up the message bit. If the message bit is zero (or one), check if $x_1 > x_2$ otherwise swap x_1 and x_2 . Do the reverse operation for the message bit one (zero)?
 - 0 For decoding, select the pixels using the same pseudo-random sequence. Check if the 2 pixels are within the pre-specified range α . If $x_1 > x_2$, the message bit is zero (one) otherwise the message bit is one (zero).
-

This scheme preserves the first order statistic (histogram) inherently without applying separate restoration process. This scheme also does not add any visual distortion to the image since the threshold used for swapping of pixels is kept considerably small ($\alpha \leq 5$) which only affects the least significant bit planes of an image. To measure the distortion introduced by the embedding in the cover image, the Peak Signal to Noise Ratio (PSNR) after embedding was observed for one hundred images. It was found that the PSNR is constantly above 46 dB as seen in Fig 1(a). This scheme is able to resist the targeted attacks based on the first order statistic proposed as shown in Fig 1(b). It should be noted that the attacks proposed are similar and hence we tested the performance of Pixel Swapping against the attack proposed only with the motivation that if it can resist the attack, then it would be able to resist the attack proposed as well.

One thing should be noted that the proposed scheme cannot ensure high embedding rates since the cover signal might have high spatial frequency thus reducing the number of pixel pairs satisfying the embedding condition. Another important point to be noted about the proposed scheme is that though it preserves first order statistics of the cover signal, it does alter the second and higher order statistics of the cover. Hence this scheme can be detected using the Blind Steganalytic methods described. These attacks use very high order statistics of the cover signal and hence these are very sensitive to even slight changes in the cover signal. Even a small amount of distortion can be detected using these blind schemes. In the next section a new statistical restoration technique has been proposed which can be used for preserving the histogram (first order statistics) of the cover signal after embedding.



(a)



(b)

Fig1.1.2.1 (a) PSNR observed for 100 images after Pixel Swapping embedding with maximum embedding rate. (b) Performance of Pixel Swapping against Sample Pair Attack proposed.

1.1.2.2 New Statistical Restoration Scheme:

In this section we introduce a new scheme for statistical restoration which can be used for repairing the artifacts which get introduced in the cover signal. This scheme tries to overcome the shortcomings of the Solanki et.al's scheme. The proposed scheme is based on the idea of pixel swapping introduced above. The cover pixels are categorized into two streams, one is for embedding and another is for restoration. At the time of embedding value of a pixel (say α , from embedding stream) is changed to β . Now the idea is to find a pixel with value β in compensation stream and change it to α . But problem with this formulation is that at the time of embedding some pixels with value β may get changed to α . So there is no need of compensation at all. To overcome this problem, at the time of embedding we maintain a record of the pairs of pixel values which get changed into one another. So, after embedding we can get an exact count of the number of pixels which have to be compensated in order to maintain the first order statistics of the cover image.

Next important point to be noted is how much distortion is added to the cover due to the compensation procedure. This distortion is somewhat dependent on how much maximum change is made per embedding by the steganographic algorithm. In our experiments, we have used the ± 1 embedding. So the absolute distortion per pixel due to embedding is at most 1. So during the compensation step, the bin value is repaired using modification of immediate neighbors (immediate left or immediate right) of that bin, satisfying lowest mean square error due to

compensation methods. The proposed method of compensation can easily be extended for $\pm k$ embedding or for that matter any kind of embedding procedure either in the spatial or the transform domain. But the amount of noise added due to compensation will increase with the increase in the noise added during the embedding step.

With regards to the efficiency of restoration process, the proposed restoration scheme is compared against the existing scheme proposed. It was observed that the proposed scheme is able to achieve better histogram restoration especially for non-Gaussian cover distributions as can be seen in Fig. 2.

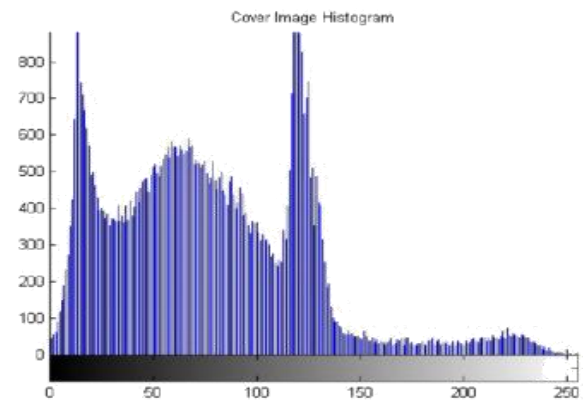
The cover image has a non-Gaussian distribution (Fig. 2(b)) and the difference histogram between the cover and stego image before compensation is shown in Fig. 2(c). Fig. 2(d) shows the difference histogram after compensation using scheme proposed by Solanki. et. al. and Fig. 2(e) shows the difference histogram after compensation using the proposed scheme. It can be seen that the proposed scheme provides better restoration than the Solanki.et.al scheme. The detailed analysis and proof of minimum noise of the proposed algorithm will be provided in the final thesis.

1.1.2.3 Security Analysis

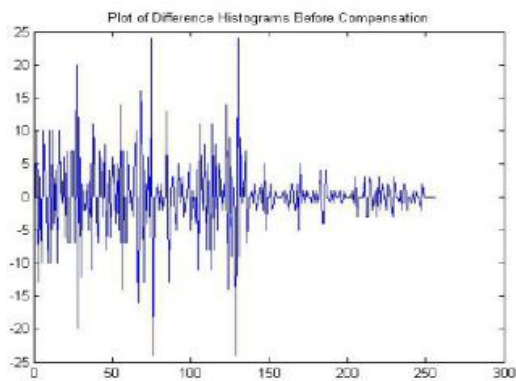
To evaluate the security of the two proposed schemes Pixel Swapping and Statistical Restoration, we have used one targeted attack called Sample Pair Analysis and one blind attack called Wavelet Moment Analyzer (WAM) proposed by Fridrich et. al. The details of these attacks can be found and the performance of the proposed schemes will be covered in detail in the thesis. It was observed that both the proposed schemes were able to resist Sample Pair attacks successfully but did not give promising results against the WAM This was attributed to the fact that although we were able to restore the marginal statistics, but we introduced additional noise to the cover in this process. The complete Results will be provided in the final thesis.



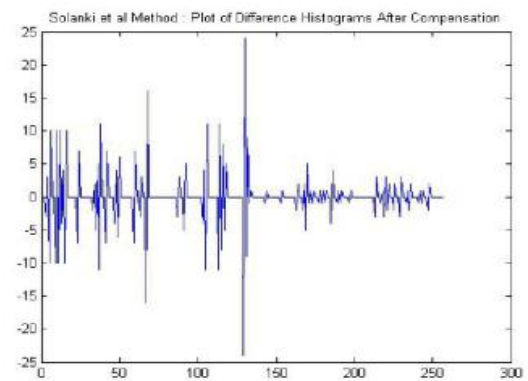
(a) Cover Image



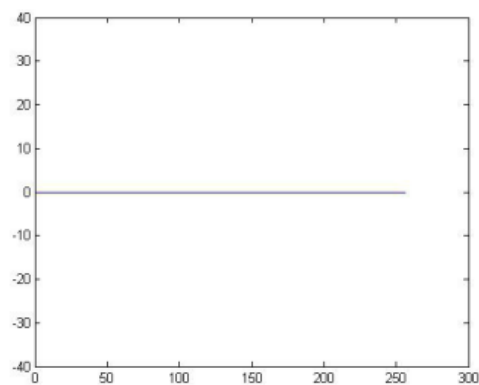
(b) Cover Image Histogram



(c) Difference Histogram Before Compensation



(d) Difference Histogram after compensation
using Solanki's scheme



(e) Difference Histogram after compensation using Proposed Scheme

Fig.1.1.2. Performance Comparison of the proposed restoration scheme and the Solanki. et. al's scheme

In this part of the thesis, we studied some of the existing blind attacks in steganalysis literature and analyze their strengths and limitations. Blind attacks often try to estimate the cover image statistics from the stego image by nullifying the changes made in the cover signal by the embedding process. Then powerful machine learning techniques are used to classify an image as cover or stego based on these statistical features.

We specifically studied the “self-calibration” based blind attacks proposed. These attacks are specifically designed to detect the presence of steganography in JPEG compressed images and are some of the most successful attacks for breaking JPEG steganography. We analyzed the performance of these attacks using a *Statistical Hypothesis Testing framework*.

The tests were performed with testing two main goals:

- 1) Test the sensitivity of the features used in the calibration attacks.
- 2) Test the effectiveness of the self-calibration process.

The problem was reduced to testing whether two given samples are drawn from same population or not. The Rank Sum (Mann-Whitney Wilcoxon Test) provided by statistical toolbox of MATLAB v7.1 was used for conducting the test and the *p-values* were observed. The hypothesis testing was performed for features extracted for the attacks. It should be pointed out that the two attacks extract features in different ways and hence the problems of hypothesis testing were formulated in separately. For the first test of testing the feature sensitivity, it was observed that as the embedding rate was increased the *p-values* obtained decreased indicating that the two populations were statistically moving further away with respect to the features obtained by the two attacks.

We also studied some of the existing approaches proposed for defeating the calibration attacks and analyzed their strengths and weaknesses. We propose a new framework called Spatial Desynchronization which tries to disturb the prediction process of the blind attack. The key feature of this framework is that it embeds data in a spatially desynchronized version of the cover image. At the time of steganalysis, the steganalyst uses the portion of the image where data is hidden for predicting the cover image statistics and hence is not able to differentiate between the cover image and the stego image. This framework is then extended to a new steganographic scheme called *Spatially Desynchronized*

Steganographic Algorithm (SDSA). The main advantage of the proposed algorithm over some of the existing schemes is that it can achieve higher embedding rates than the existing schemes while maintaining same levels of security. The proposed algorithm is analyzed in the light of the statistical framework proposed above. It is shown that proposed algorithm produce stego images which are statistically closer to the cover image population. The robustness of the proposed algorithm is also tested against the calibration attacks and it is shown that the proposed algorithm could embed more payload into the cover signal while providing better levels of security. To check the performance of the *SDSA* algorithm against blind attacks not based on self-calibration, the performance was also tested against the attack proposed. The detailed results and analysis of this work will be provided in the final thesis.

1.2 Problem Statement:

The former consists of linguistic or language forms of hidden writing. The later, such as invisible ink, try of hide messages physically. One disadvantage of linguistic steganography is that users must equip themselves to have a good knowledge of industry. In recent years, everything is trending toward digitization. And with the development of the internet technology, digital media can be transmitted conveniently over the network. Therefore, messages can be secretly carried by digital media by using the steganography techniques, and then be transmitted through the internet rapidly

Steganography is the art of hiding the fact that communication is taking place, by hiding information in other information. Many different carrier file formats can be used, but digital images are the most popular because of their frequency on the internet. For hiding secret information in images, there exists a large variety of steganography techniques some are more complex than others and all of them have respective strong and weak points.

So we prepare this application, to make the information hiding more simple and user friendly

1.3 Project Overview

The word steganography comes from the Greek “Stegano”, which mean covered or secret and – “graphy” mean writing or drawing. Therefore, steganography mean, literally, covered writing. It is the art and science of hiding information such its presence cannot be detected and a

communication is happening. A secret information is encoding in a manner such that the very existence of the information is concealed. Paired with existing communication methods, steganography can be used to carry out hidden exchanges.

The main goal of this projects it to communicate securely in a completely undetectable manner and to avoid drawing suspicion to the transmission of a hider data. There has been a rapid growth of interest in steganography for two reasons:

The publishing and broadcasting industries have become interested in techniques for hiding encrypted copyright marks and serial numbers in digital films, audio recordings, books and multimedia products

Moves by various governments to restrict the availability of encryption services have motivated people to study methods by which private messages can be embedded in seemingly innocuous cover messages.

The basic model of steganography consists of Carrier, Message and password. Carrier is also known as cover-object, which the message is embedded and serves to hide the presence of the message.

Basically, the model for steganography is shown on following figure:

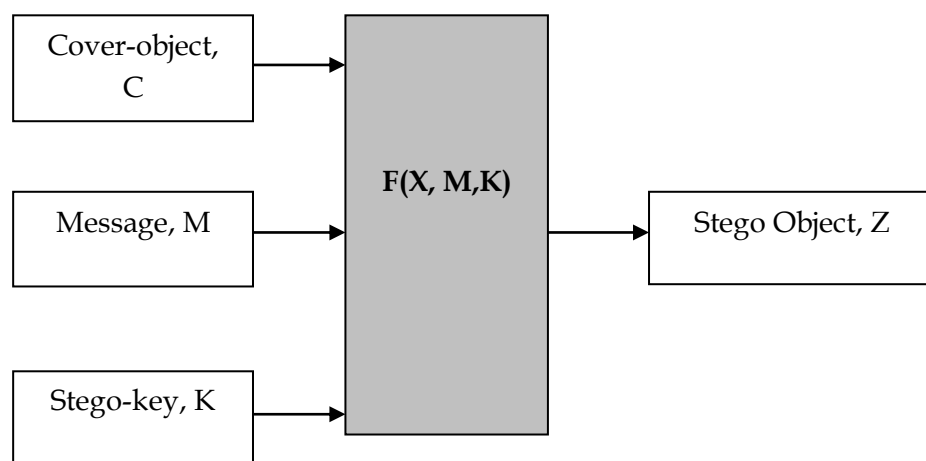


Fig. 1.3 the model for steganography

Message is the data that the sender wishes to remain confidential. It can be plain text, cipher text, other image, or anything that can be embedded in a bit stream such as a copyright mark, a covert communication, or a serial number. Password is known as *stego-key*, which ensures that only recipient who know the corresponding decoding key will be able to extract the message from a *cover-object*. The *cover-object* with the secretly embedded message is then called the *Stego-object*.

Recovering message from a *stego-object* requires the *cover-object* itself and a corresponding decoding key if a *stego-key* was used during the encoding process. The original image may or may not be required in most applications to extract the message.

There are several suitable carriers below to be the *cover-object*:

- Network protocols such as TCP, IP and UDP
- Audio that using digital audio formats such as wav, midi, avi, mpeg, mp4 and voc
- File and Disk that can hides and append files by using the slack space
- Text such as null characters, just alike morse code including html and java
- Images file such as bmp, gif and jpg, where they can be both color and gray-scale.

In general, the information hiding process extracts redundant bits from *cover-object*. The process consists of two steps:

- Identification of redundant bits in a *cover-object*. Redundant bits are those bits that can be modified without corrupting the quality or destroying the integrity of the *cover-object*.
- Embedding process then selects the subset of the redundant bits to be replaced with data from a secret message. The *stego-object* is created by replacing the selected redundant bits with message bits

1.4 Project Objective

The goal of steganography is covert communication. So, a fundamental requirement of this steganography system is that the hidden message carried by stego-media should not be sensible to human beings.

The other goal of steganography is to avoid drawing suspicion to the existence of a hidden message. This approach of information hiding technique has recently become important in a number of application areas.

This project has following objectives:

- To produce security tool based on steganography techniques.
- To explore techniques of hiding data using encryption module of this project
- To extract techniques of getting secret data using decryption module.

Steganography sometimes is used when encryption is not permitted. Or, more commonly, steganography is used to supplement encryption. An encrypted file may still hide information using steganography, so even if the encrypted file is deciphered, the hidden message is not seen.

1.5 Project Scope:

This project is developed for hiding information in any image file. The scope of the project is implementation of steganography tools for hiding information includes any type of information file and image files and the path where the user wants to save Image and extracted file.

1.6 History of Steganography:

Throughout history Steganography has been used to secretly communicate information between people.

Some examples of use of Steganography in past times are:

1. During World War 2 invisible ink was used to write information on pieces of paper so that the paper appeared to the average person as just being blank pieces of paper. Liquids such as milk, vinegar and fruit juices were used, because when each one of these substances are heated they darken and become visible to the human eye.

2. In Ancient Greece they used to select messengers and shave their head, they would then write a message on their head. Once the message had been written the hair was allowed to grow back. After the hair grew back the messenger was sent to deliver the message, the recipient would shave off the messenger's hair to see the secret message.
3. Another method used in Greece was where someone would peel wax off a tablet

1.7 Definition, Acronyms and Abbreviations

- C#, .Net framework 3.5
- DB-Database (Storage Source)
- OS-Operating System-Windows

- **Limitations of the Software:**

This project has an assumption that is both the sender and receiver must have shared some secret information before imprisonment. Pure steganography means that there is none prior information shared by two communication parties.

1.8 System Analysis & Design

Steganography system requires any type of image file and the information or message that is to be hidden. It has two modules encrypt and decrypt.

Microsoft .Net framework prepares a huge amount of tool and options for programmers that they simplify programming. One of .Net tools for pictures and images is auto-converting most types of pictures to BMP format. I used this tool in this software called "Steganography" that is written in C#.Net language and you can use this software to hide your information in any type of pictures without any converting its format to BMP (software converts inside it).

The algorithm used for Encryption and Decryption in this application provides using several layers' lieu of using only LSB layer of image. Writing data starts from last layer (8th or LSB layer); because significant of this layer is least and every upper layer has doubled significant from its down layer. So every step we go to upper layer image quality decreases and image retouching transpires.

The encrypt module is used to hide information into the image; no one can see that information or file. This module requires any type of image and message and gives the only one image file in destination.

The decrypt module is used to get the hidden information in an image file. It takes the image file as an output, and give two file at destination folder, one is the same image file and another is the message file that is hidden it that.

Before encrypting file inside image we must save name and size of file in a definite place of image. We could save file name before file information in LSB layer and save file size and file name size in most right-down pixels of image. Writing this information is needed to retrieve file from encrypted image in decryption state.

The graphical representation of this system is as follows:

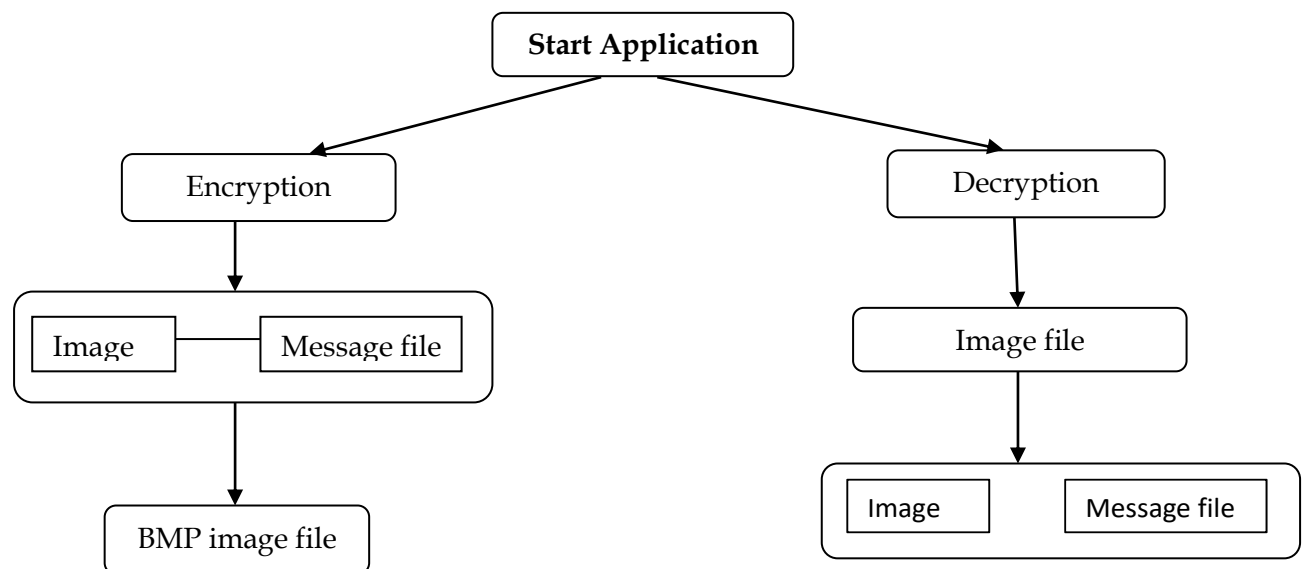


Fig. 1.8 Graphical representation of the system

Chapter 2

Methodology

User needs to run the application. The user has two tab options – encrypt and decrypt. If user select encrypt, application give the screen to select image file, information file and option to save the image file. If user select decrypt, application gives the screen to select only image file and ask path where user want to save the secrete file.

This project has two methods – Encrypt and Decrypt.

In encryption the secret information is hiding in with any type of image file.

Decryption is getting the secret information from image file.

2.1 Detecting Steganography:

The art of detecting Steganography is referred to as **Steganalysis**.

To put is simply Steganalysis involves detecting the use of Steganography inside of a file. Steganalysis does not deal with trying to decrypt the hidden information inside of a file, just discovering it.

There are many methods that can be used to detect Steganography such as:

“Viewing the file and comparing it to another copy of the file found on the Internet (Picture file). There are usually multiple copies of images on the internet, so you may want to look for several of them and try and compare the suspect file to them. For example, if you download a JPED and your suspect file is also a JPED and the two files look almost identical apart from the fact that one is larger than the other, it is most probable you suspect file has hidden information inside of it.

2.2 Stego Tools:

Based on open-source platforms, list of various steganography tools are used to enhance their image quality and performance related issues:

Matroschka, Openpuff, Pngchek, SilenteYE, Stepic etc. for Linux distros

Snow, Camouflage, S-tools etc. for windows packages.

2.3 Steganography vs Cryptography:

Basically, the purpose of cryptography and steganography is to provide secret communication. However, steganography is not the same as cryptography. Cryptography hides the contents of a secret message from malicious people, whereas steganography even conceals the existence of the message. In cryptography, the system is broken when the attacker can read the secret message. Breaking a steganography system needs the attacker to detect that steganography has been used.

It is possible to combine the techniques by encrypting a message using cryptography and then hiding the encrypted message using steganography. The resulting stego-image can be transmitted without revealing that secret information is being exchanged.

2.4 Steganography vs Watermarking:

Steganography pays attention to the degree of invisibility while watermarking pays most of its attribute to the robustness of the message and its ability to withstand attacks of removal, such as image operations (rotation, cropping, filtering), audio operations (rerecording, filtering) in the case of images and audio files being watermarked respectively.

It is a non-questionable fact that the detectability of a vessel with an introduced data (steganographic message or a watermark) is a function of the changeability function of the algorithm over the vessel.

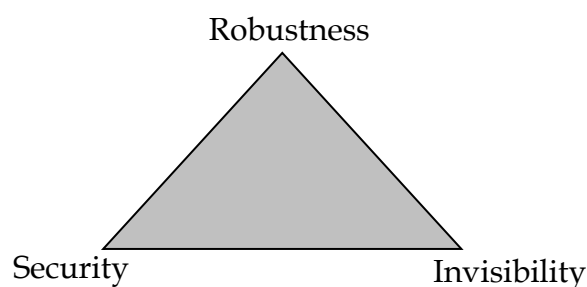


Fig. 2.4 Detectability of a vessel with an introduced data

That is the way the algorithm changes the vessel and the severity of such an operation determines without doubt the detectability of the message, since detectability is a function of file

characteristics deviation from the norm, embedding operation attitude and change severity of such change decides vessel file delectability.

A typical triangle of conflict is message Invisibility, Robustness, and Security. Invisibility is a measure of the in notability of the contents of the message within the vessel.

Security is sinominous to the cryptographic idea to message security, meaning inability of reconstruction of the message without the proper secret key material shared.

Robustness refers to the endurance capability of the message to survive distortion or removal attacks intact. It is often used in the watermarking field since watermarking seeks the persistence of the watermark over attacks, steganographic messages on the other hand tend to be of high sensitivity to such attacks. The more invisible the message is the less secure it is (cryptography needs space) and the less robust it is (no error checking/recovery introduced). The more robust the message is embedded the more size it requires and the more visible it is.

2.5 Steganography Techniques:

Over the past few years, numerous steganography techniques that embed hidden messages in multimedia objects have been proposed. There have been many techniques for hiding information or messages in images in such a manner that alteration made to the image is perceptually indiscernible. Commonly approaches are including LSB, Masking and filtering and Transform techniques.

2.5.1 LSB

Least significant bit (LSB) insertion is a simple approach to embedding information in image file. The simplest steganography techniques embed the bits of the message directly into least significant bit plane of the cover-image in a

deterministic sequence. Modulating the least significant bit does not result in human perceptible difference because the amplitude of the change is small. In this technique, the embedding capacity can be increased by using two or more least significant bits. At the same time, not only the risk of making the embedded message statistically detectable increase but also the image fidelity degrades. Hence a variable size LSB embedding schema is presented, in which the number of

LSBs used for message embedding/extracting depends on the local characteristics of the pixel. The advantage of LSB-based method is easy to implement and high message pay-load.

Although LSB hides the message in such way that the humans do not perceive it, it is still possible for the opponent to retrieve the message due to the simplicity of the technique. Therefore, malicious people can easily try to extract the message from the beginning of the image if they are suspicious that there exists secret information that was embedded in the image.

2.5.2 SIHS

A system named Secure Information Hiding System (SIHS) is proposed to improve the LSB scheme. It overcomes the sequence-mapping problem by embedding the message into a set of random pixels, which are scattered on the cover-image.

Masking and filtering techniques, usually restricted to 24 bits and gray scale image, hide information by marking an image, in a manner similar to paper watermarks. The technique performs analysis of the image, thus embed the information in significant areas so that the hidden message is more integral to cover image than just hiding it in the noise level.

Transform techniques embed the message by modulating coefficient in a transform domain, such as the Discrete Fourier Transform, or Wavelet Transform. These methods hide messages in significant areas of the cover image, which make them more robust to attack. Transformations can be applied over the entire image, to block throughout the image, or other variant.

2.5.3 Image Steganography and bitmap pictures:

Using bitmap pictures for hiding secret information is one of most popular choices for Steganography. Many types of software built for this purpose, some of these software use password protection to encrypting information on picture. To use these software, you must have a 'BMP' format of a pictures to use it, but using other type of pictures like "JPEG", "GIF" or any other types is rather or never used, because of algorithm of "BMP" pictures for Steganography is simple. Also we know that in the web most popular of image types are "JPEG" and other types not "BPM", so we should have a solution for this problem.

This software provides the solution of this problem, it can accept any type of image to hide information file, but finally it gives the only "BMP" image as an output that has hidden file inside it.



Fig. 2.5.3 Image Steganography and bitmap pictures differences

The Inputs (left) and the corresponding outputs (rights) of the program using our proposed technique for hiding data.

(a, c)—Left _normal Image.

(b, d) —Right _Filtered Image or Image with hidden Contents.

2.6 Specific Requirements:

This section contains the software requirements to a level of detail sufficient to enable designers to design the system and testers to test that system.

2.6.1 External Interface Requirements

2.6.1.1 Software Interfaces:

Front End: C# - .NET Framework 3.5

1) NetBeans

2) OS: Windows

Namespace and Base Classes used in this project are

- System.Drawing
- System.IO
- System.Deployment
- System.Xml
- System.Data

2.6.1.2 Hardware Requirements:

Processor: Preferably 1.0 GHz or Greater.

RAM : 512 MB or Greater.

HDD : min 2GB (Preferable SSD)

2.6.2 Functional Requirements:

Depending upon the user role he/she will be able to access only the specific modules of the system or application.

1) Administrative User or must have authorized access to the system.

2) User (with role basic operator) will be able to convert /modify /add/ delete/ information Contents (Output File)

2.6.3 Other Requirements

Non- Functional Requirements

1. Safety Requirements

The software may get crashed at any certain time due to virus or any infected input file or unspecified operating System failure. Therefore, it is required to take the output data backup.

2. Security Requirements

We are going to develop a software to hide any image into another file/image. Depending upon the format and category of software access rights are decided. It means if user is an administrator or have an information about the written code and tricks then he/she can be able to modify the data, delete, append etc.

3. Hardware Constraints

The system requires special permissions and storage to operate all the functions properly. In case Antivirus gives an alert for security concerns ignore or disable it (it's because of its custom written application code).

Limitations of the Software:

This project has an assumption that is both the sender and receiver must have shared some secret information before imprisonment. Pure steganography means that there is no prior information shared by two communication parties.

Chapter 3

Analysis Models

3.1 Stego Library Class

Steganography system requires any type of image file and the information or message that is to be hidden. It has two modules encrypt and decrypt. Microsoft .Net framework prepares a huge amount of tool and options for programmers that they simplify programming. One of .Net tools for pictures and images is auto-converting most types of pictures to BMP format. I used this tool in this software called “Steganography” that is written in C#.Net language and you can use this software to hide your information in any type of pictures without any converting its format to BMP (software converts inside it). The algorithm used for Encryption and Decryption in this application provides using several layers’ lieu of using only LSB layer of image. Writing data starts from last layer (8th or LSB layer); because significant of this layer is least and every upper layer has doubled significant from its down layer. So every step we go to upper layer image quality decreases and image retouching transpires.

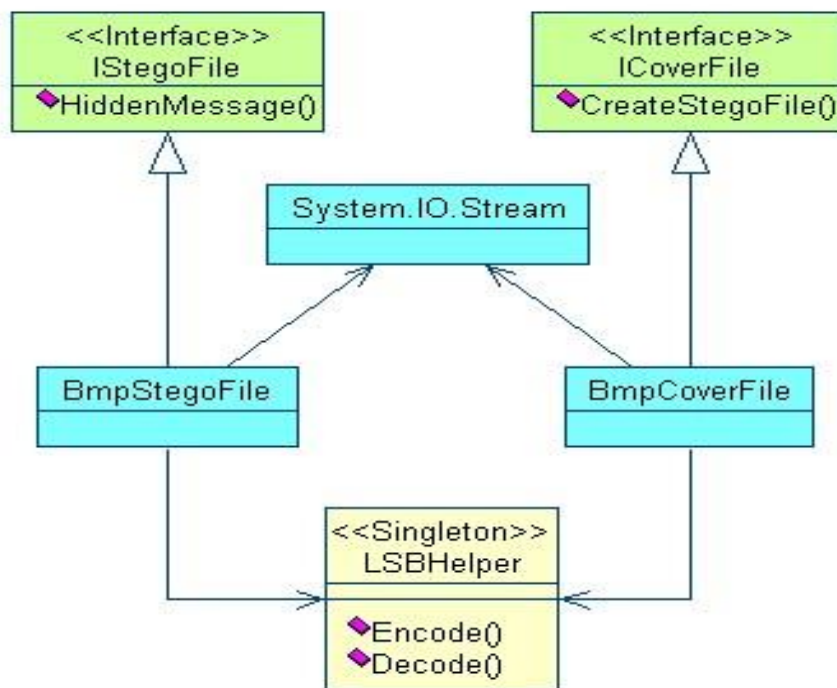


Fig. 3.1 Stego Library Class Diagram

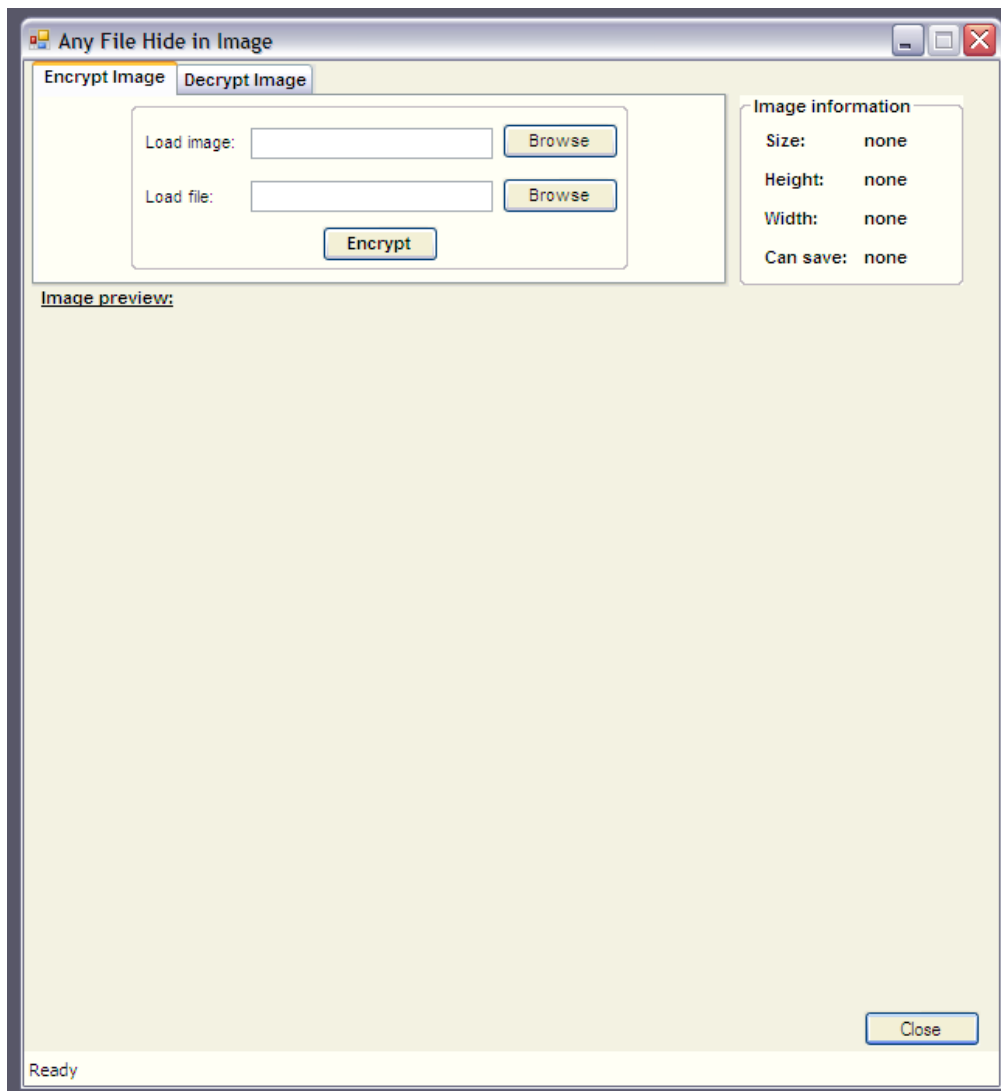
The stego library defines interfaces that support different types of cover and stego files. The **BmpCoverFile** and **BmpStegoFile** classes are concrete implementations that operate on 24-bit bitmapped images.

Chapter 4

Instructions & User Interface

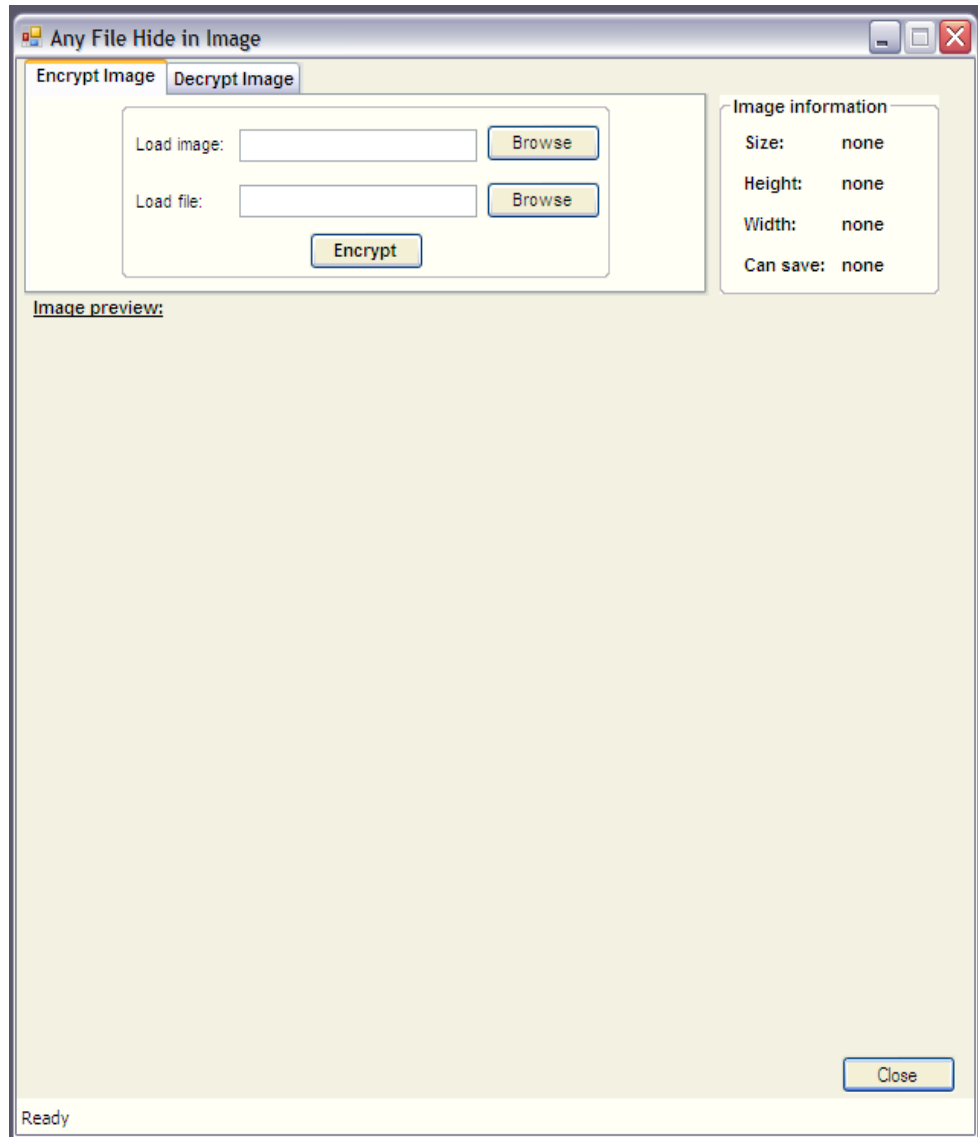
4.1 User Manual

This is the first screen which has two tab options – one is Encrypt Image for encryption and another is Decrypt image for decryption. In right – top panel is displays the information about the image such as size, height and width.

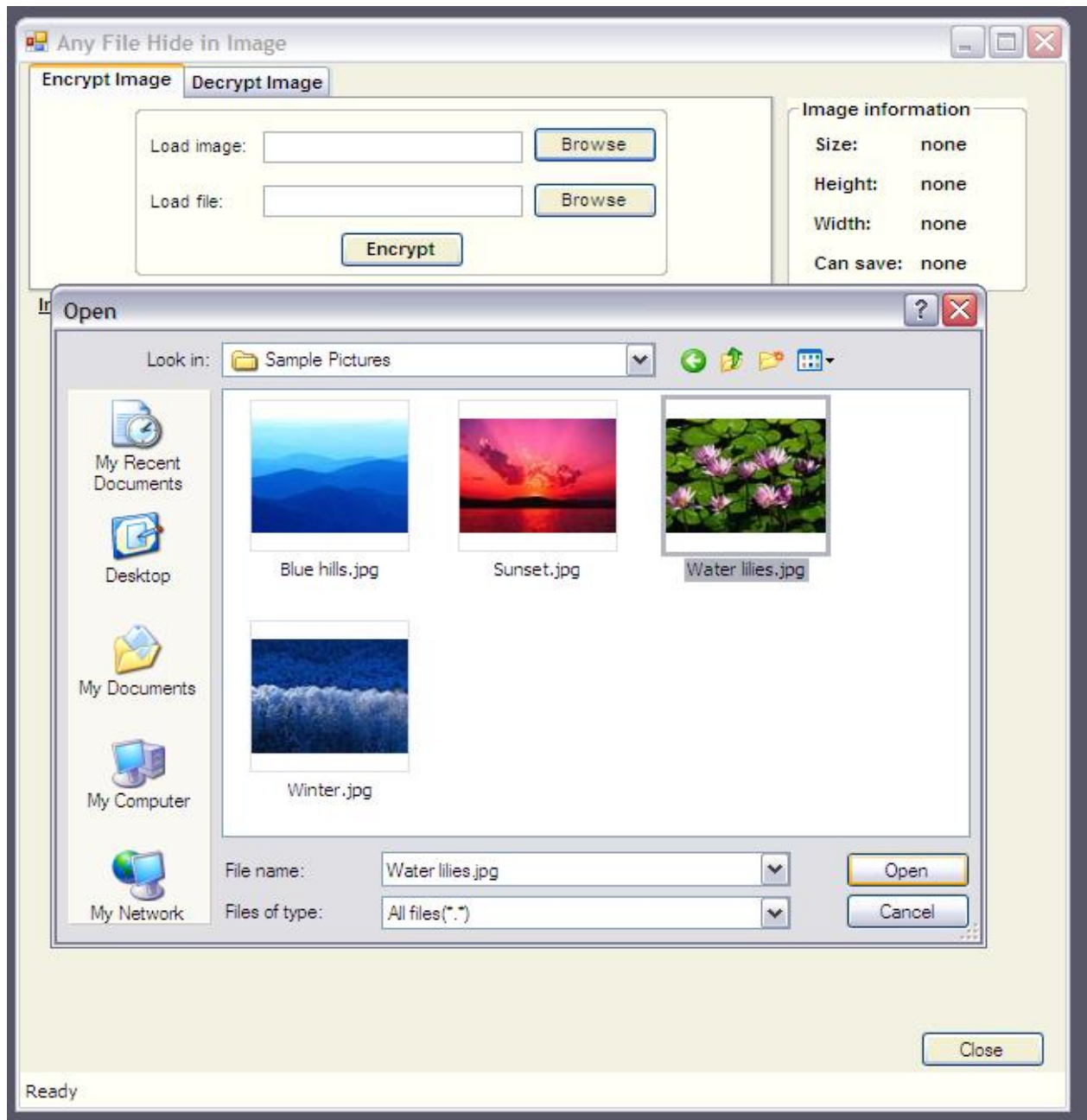


4.2 Encryption

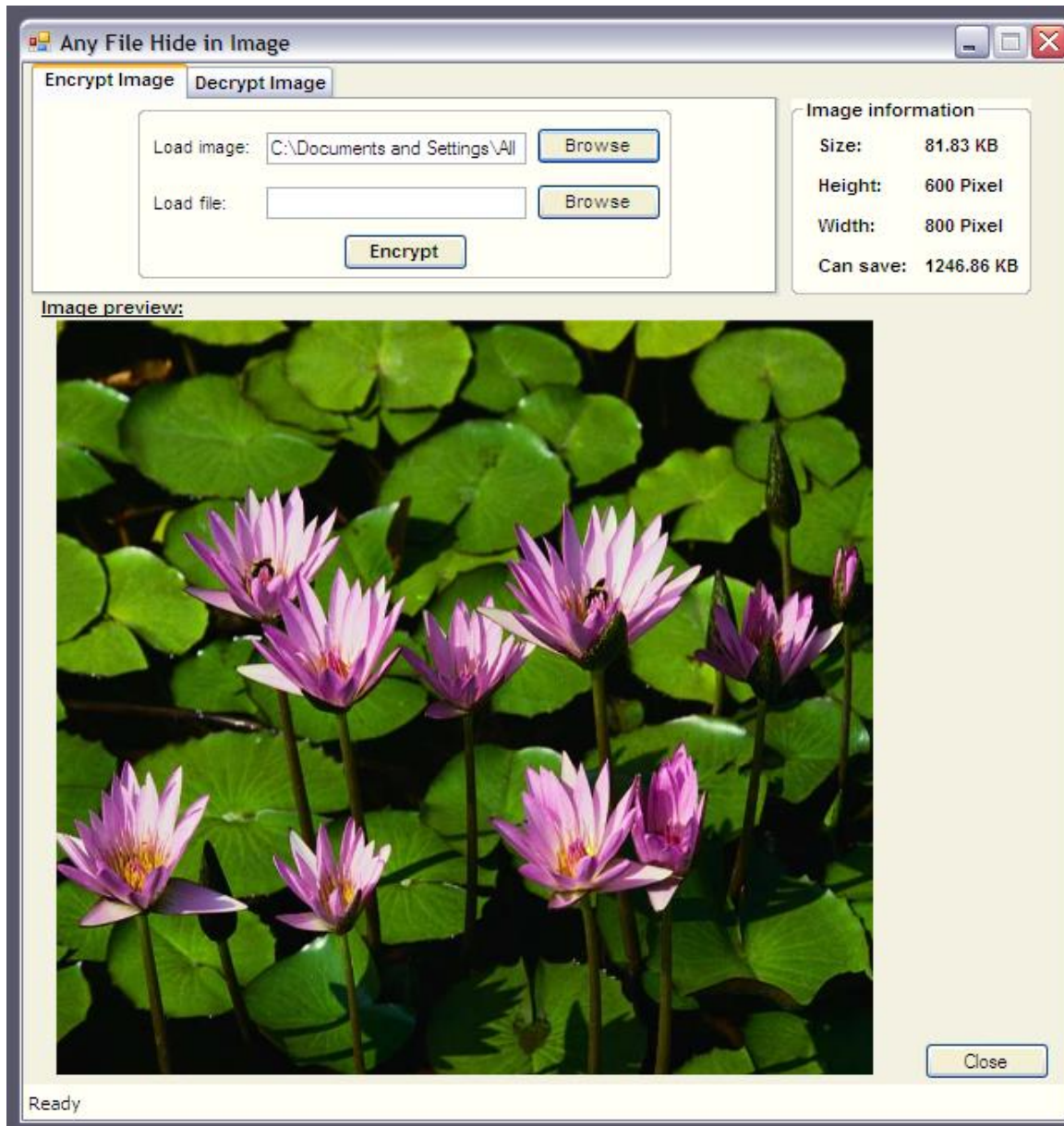
1. For Encryption select Encrypt Image tab option.



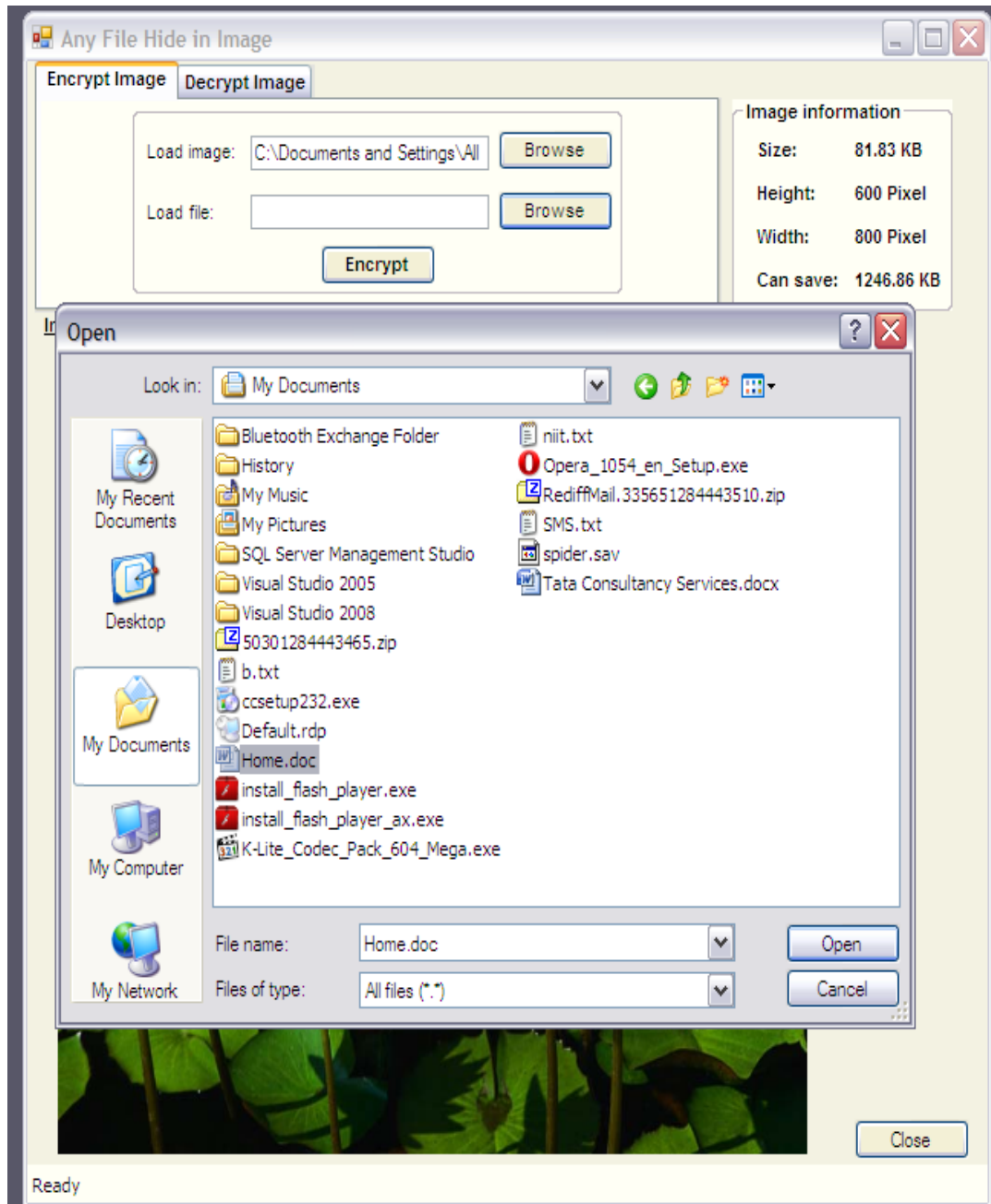
2. For load image click on button “Browse” that is next to the Load Image textbox. The file open dialog box will display as follows, select the Image file, which you want to use hide information and click on Open button.



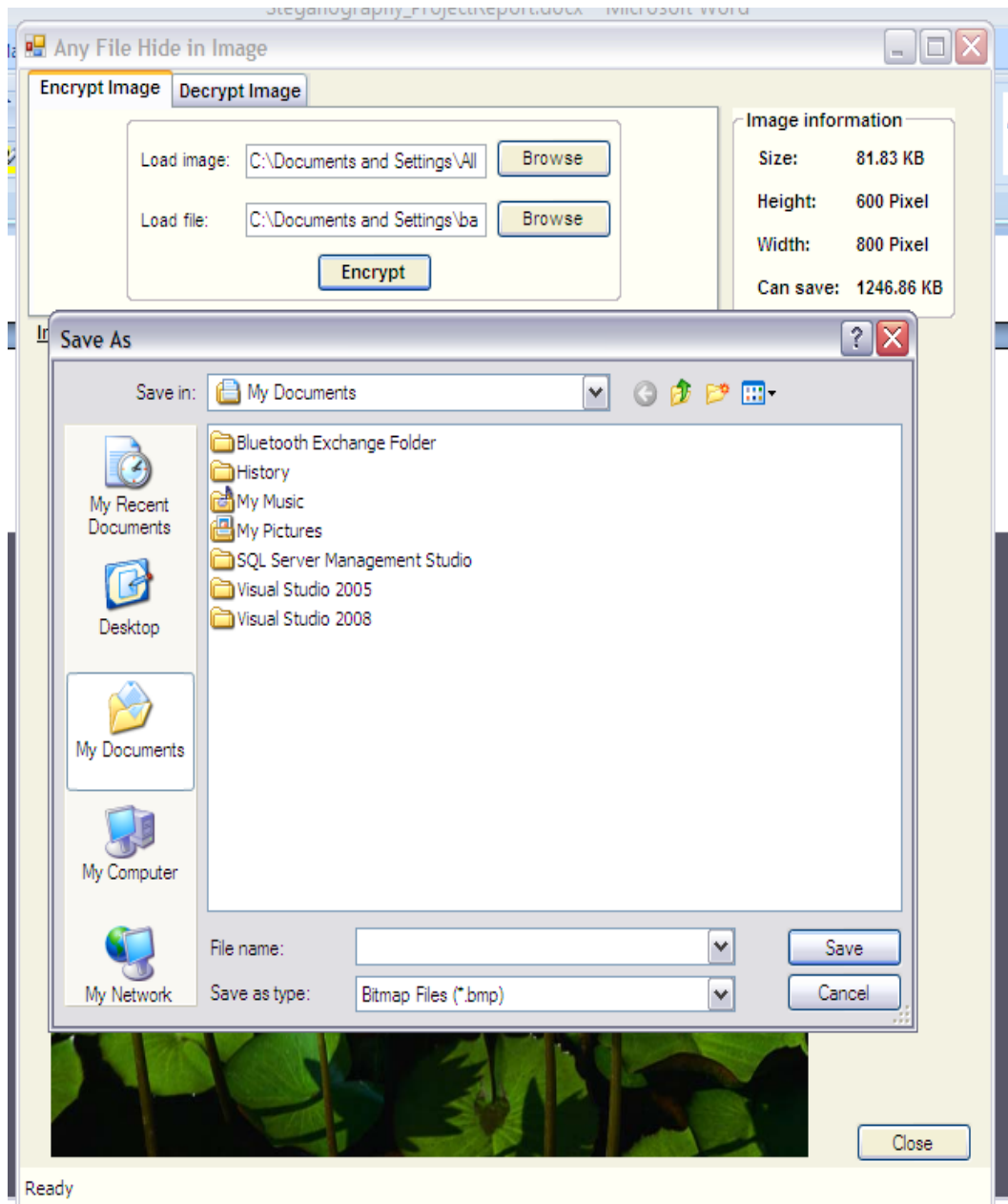
3. The image file will have opened and is displays as follows. Next, click on “Browse” button that is next to the Load File textbox.



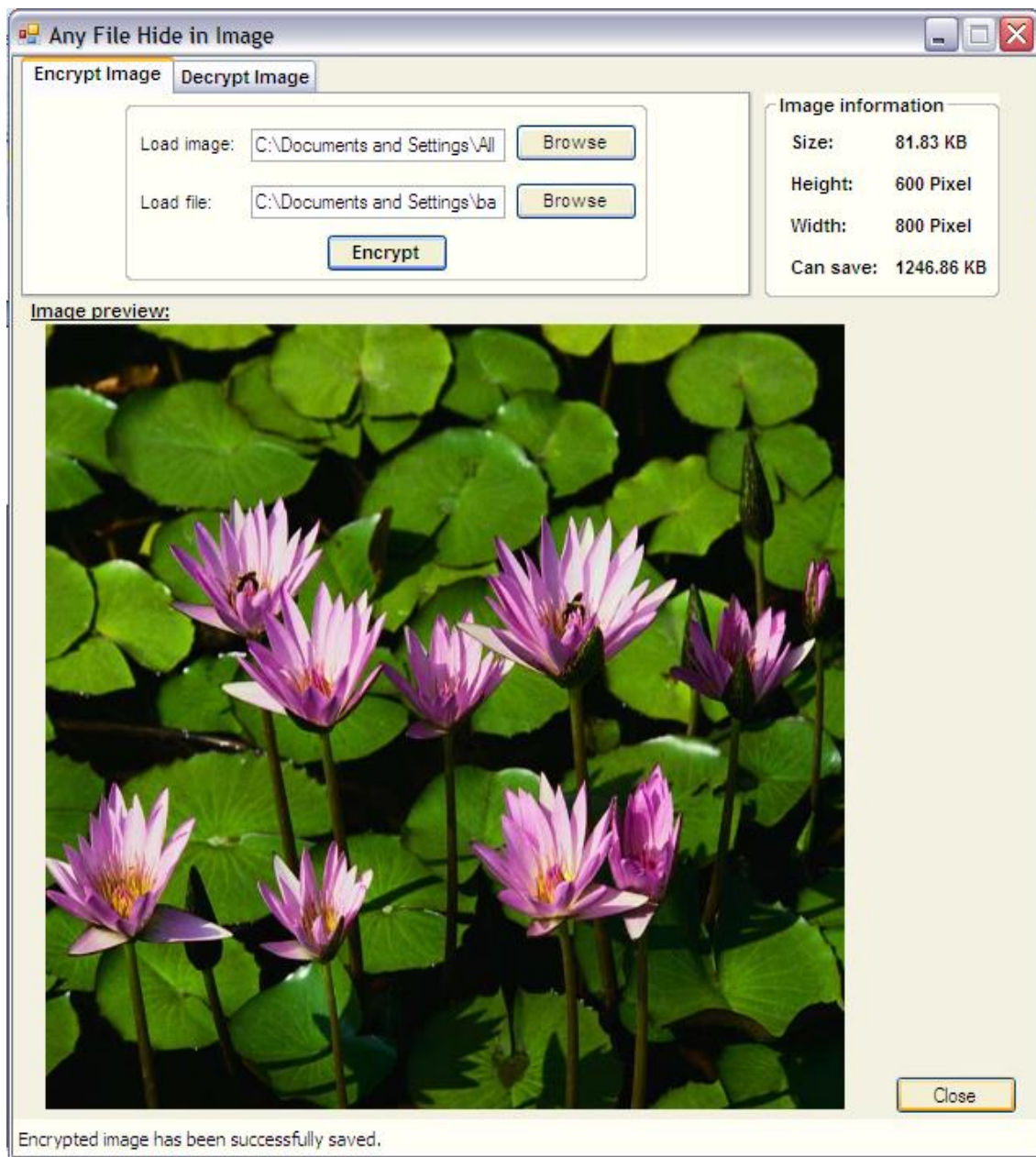
4. Again the file open dialog box will appear, select any type of file whatever you want to hide with the image and click on ok button.



5. The next step is to encrypt the file. Now click on “Encrypt” button, it will open the save dialog box which ask you to select the path to save the New image file and the Image file name. The default format of image file is BMP.

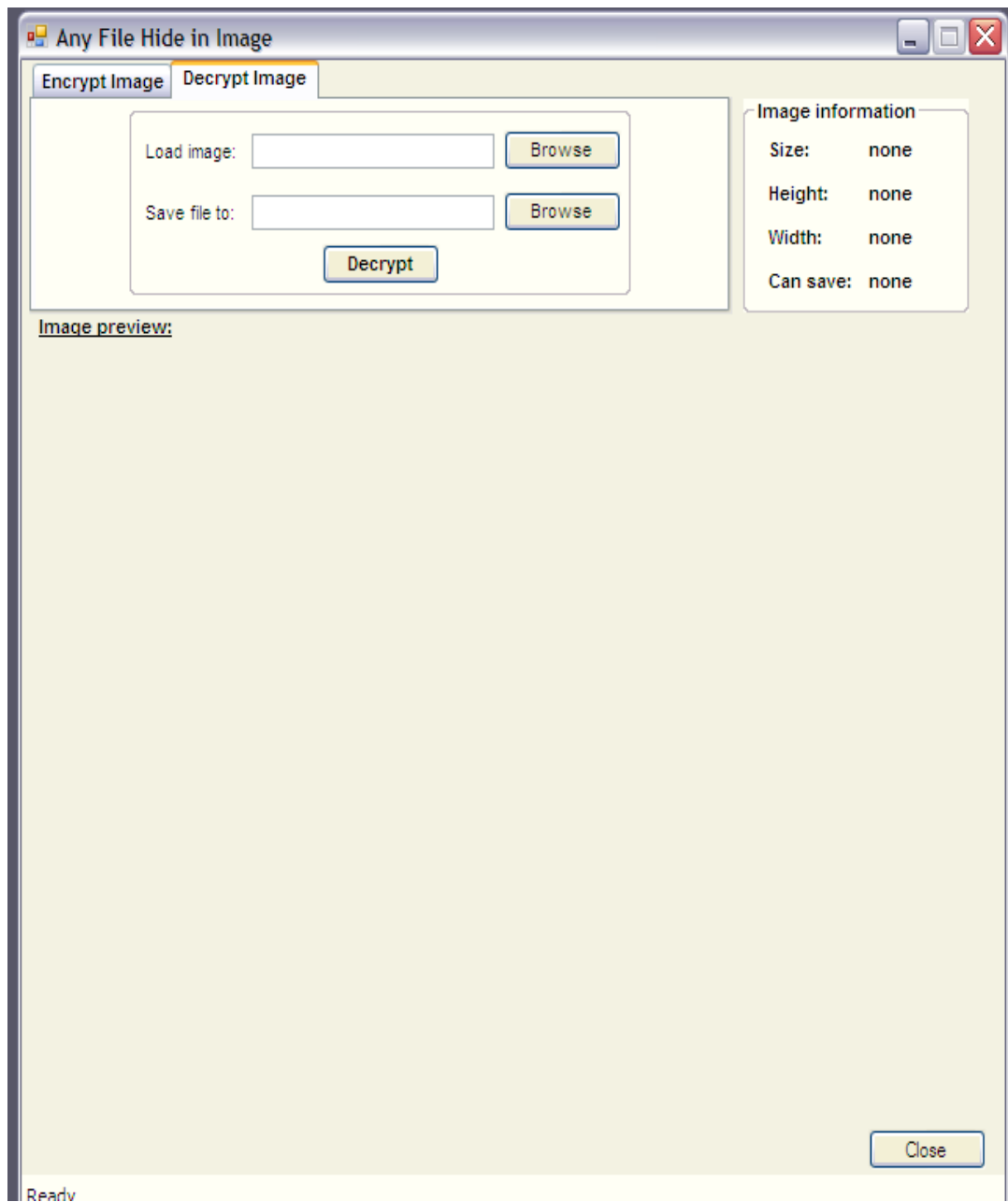


6. Final Image.

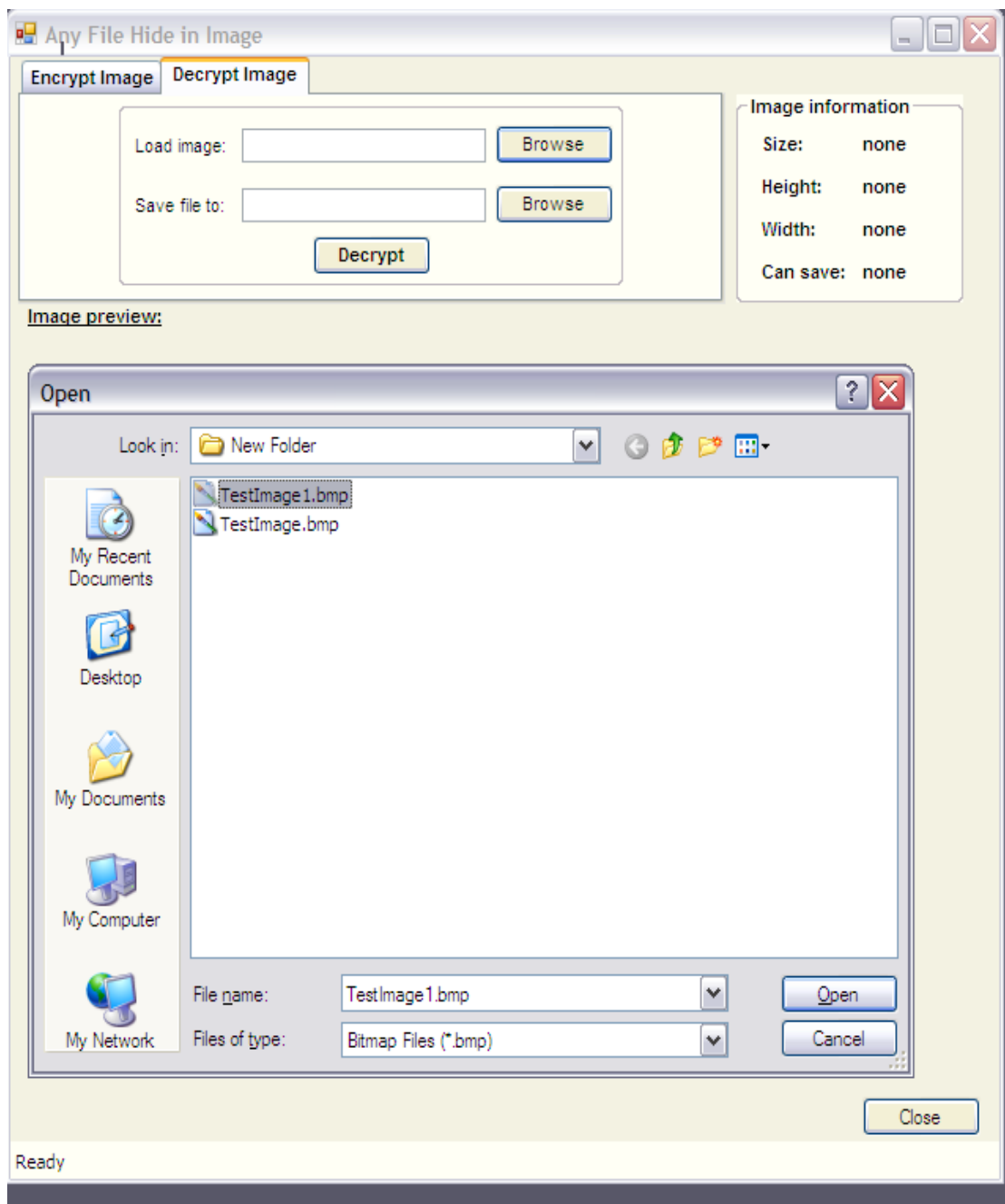


4.3 Decryption

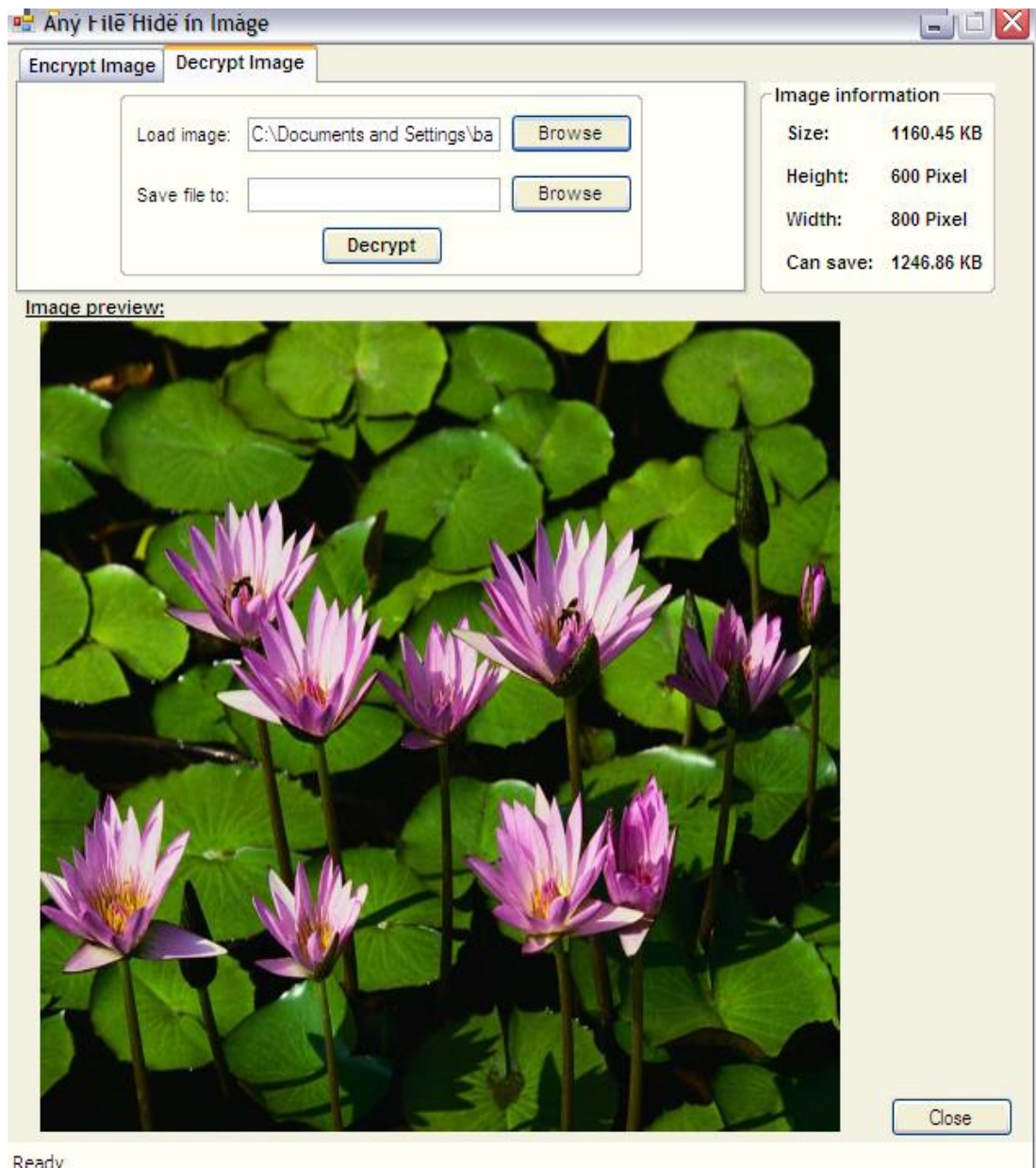
1. Select the Decryption Image tab option.



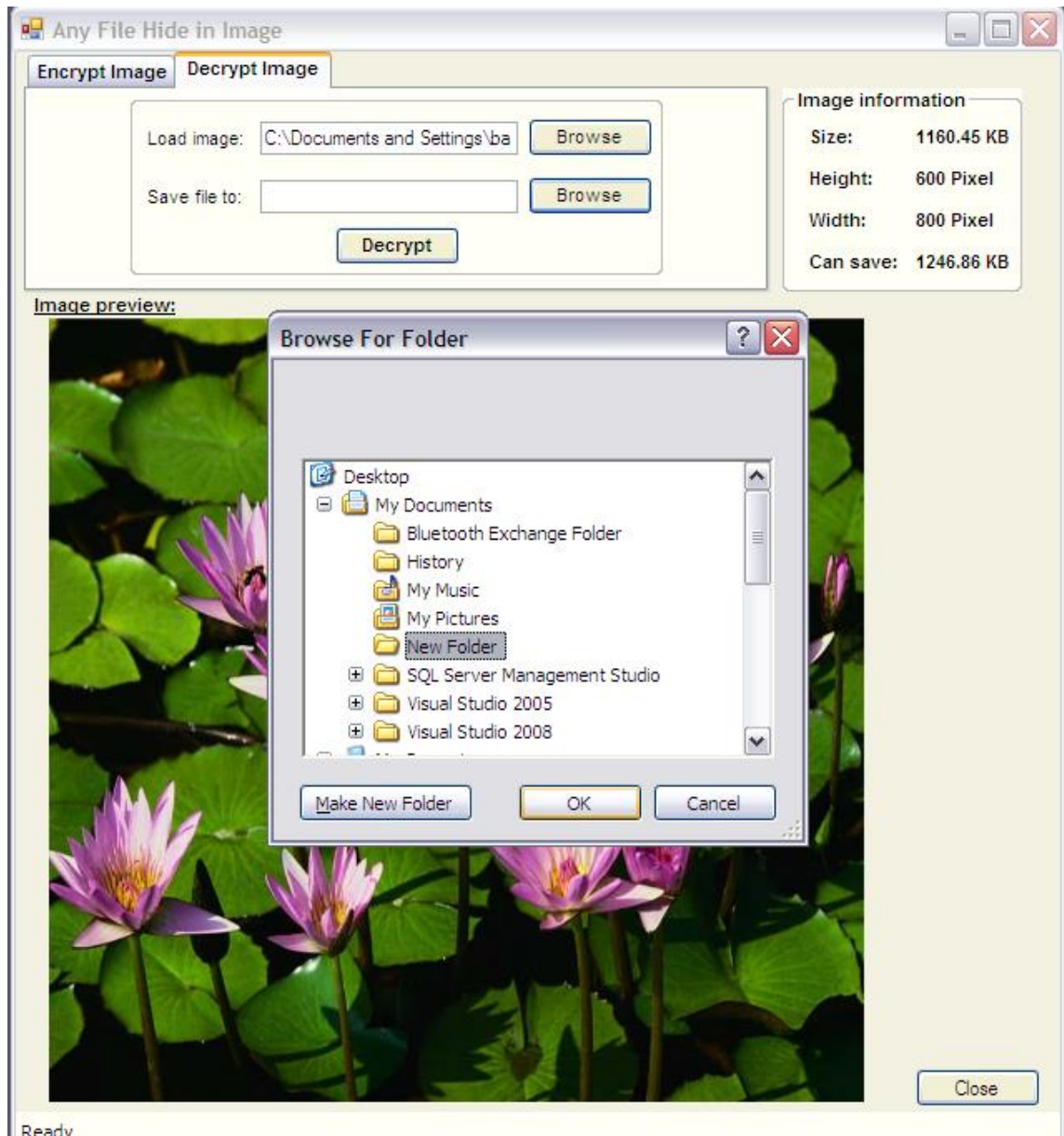
2. Next click on the “Browse” button, which open the Open file dialog box, here you have to select the image which is Encrypted and has hidden information file. Select the image file and click on Open button.



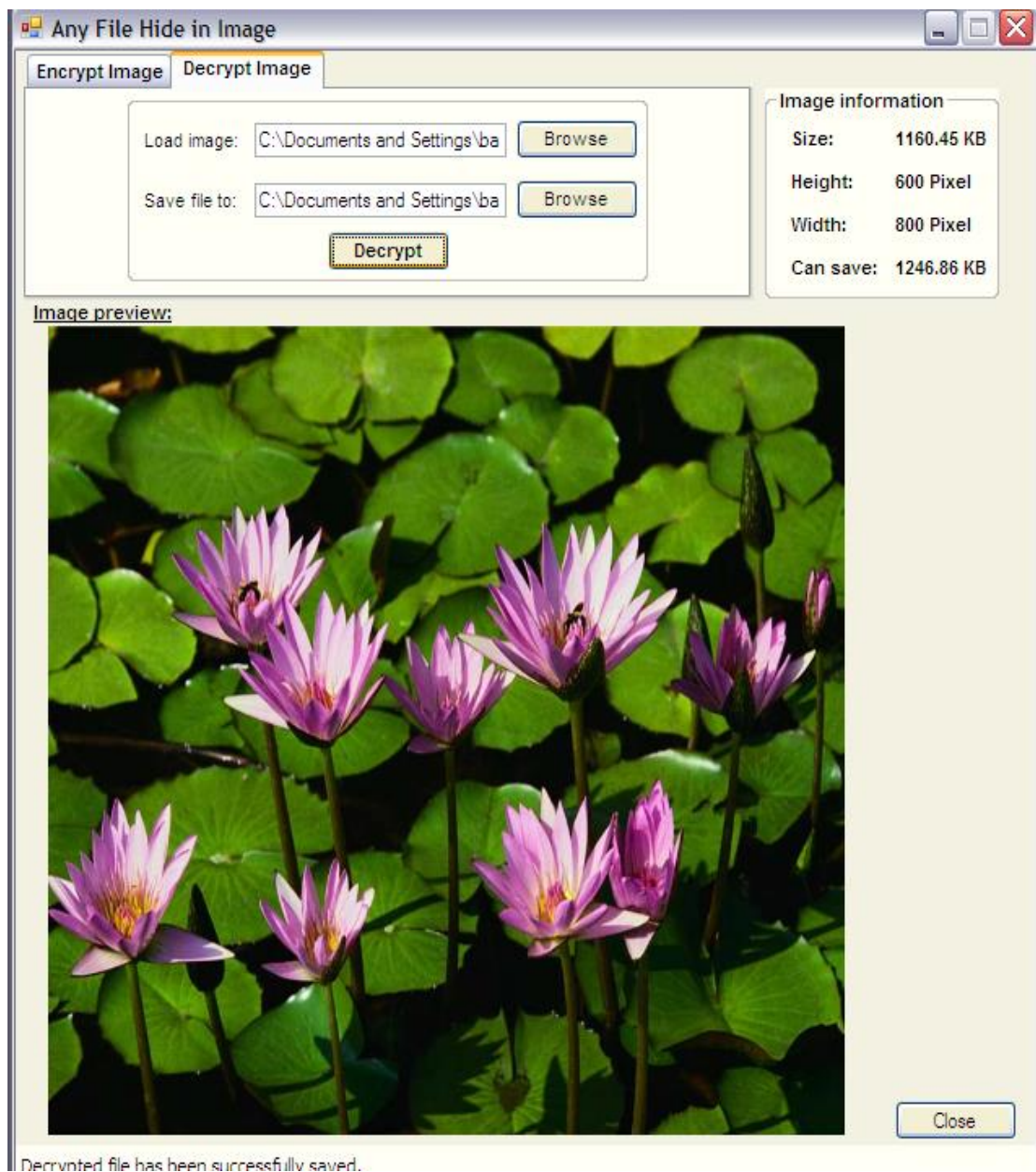
3. The image file displayed as follows:



4. Now click on “Browse” button which is next to “Save file to” textbox. It will open a dialog box that is “Browse for folder”. It asks you to select the path or folder, where you want to extract the hidden file. Select the folder and click on Ok button.



5. Now click on Decrypt button, it will decrypt the image, the hidden file and image file is saved into selected folder. The message for successful decryption is displayed on the status bar which is places at bottom of the screen.



Chapter 5

System Testing

5.1 Introduction

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the software system meets its requirements and user expectations and does not fail in an unacceptable manner.

5.2 Methods used:

5.2.1 Bitmap Steganography:

Bitmap type is the simplest type of picture because that it doesn't have any technology for decreasing file size. Structure of these files is that a bitmap image created from pixels that any pixel created from three colors (red, green and blue said RGB) each color of a pixel is one-byte information that shows the density of that color. Merging these three colors makes every color that we see in these pictures. We know that every byte in computer science is created from 8 bits that first bit is Most-Significant-Bit (MSB) and last bit Least-Significant-Bit (LSB), the idea of using Steganography science is in this place; we use LSB bit for writing our security information inside BMP pictures. So if we just use last layer (8th layer) of information, we should change the last bit of pixels, in other hands we have 3 bits in each pixel so we have $3 \times \text{height} \times \text{width}$ bits' memory to write our information. But before writing our data we must write name of data(file), size of name of data & size of data. We can do this by assigning some first bits of memory (8th layer).

(00101101	0001110 <u>1</u>	11011100)
(10100110	1100010 <u>1</u>	00001100)
(11010010	1010110 <u>0</u>	01100011)

Using each 3 pixels of picture to save a byte of data

5.2.2 Encryption Process

The **encrypt module** is used to hide information into the image; no one can see that information or file. This module requires any type of image and message and gives the only one image file in destination.

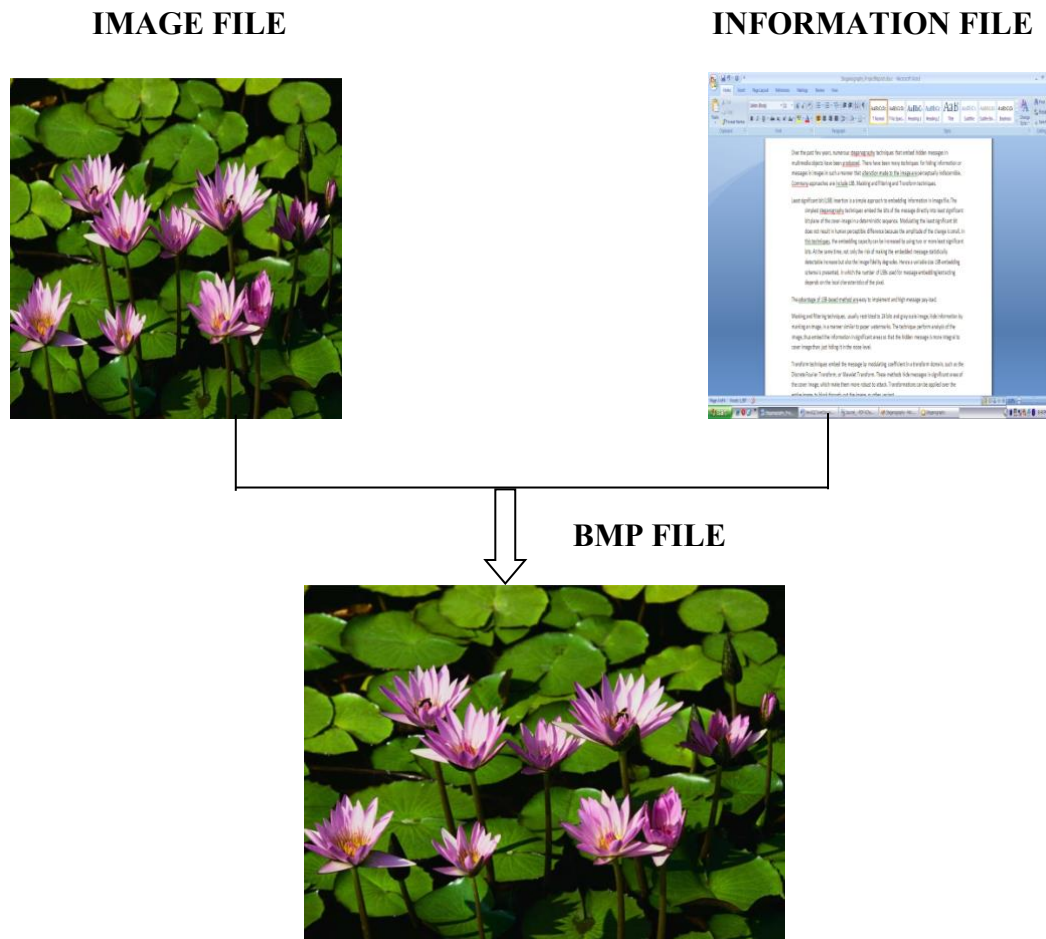


Fig. 3.2 hiding information into the image using Encryption module

5.3 Decryption Process

The **decrypt module** is used to get the hidden information in an image file. It takes the image file as an output, and give two file at destination folder, one is the same image file and another is the message file that is hidden it that file.

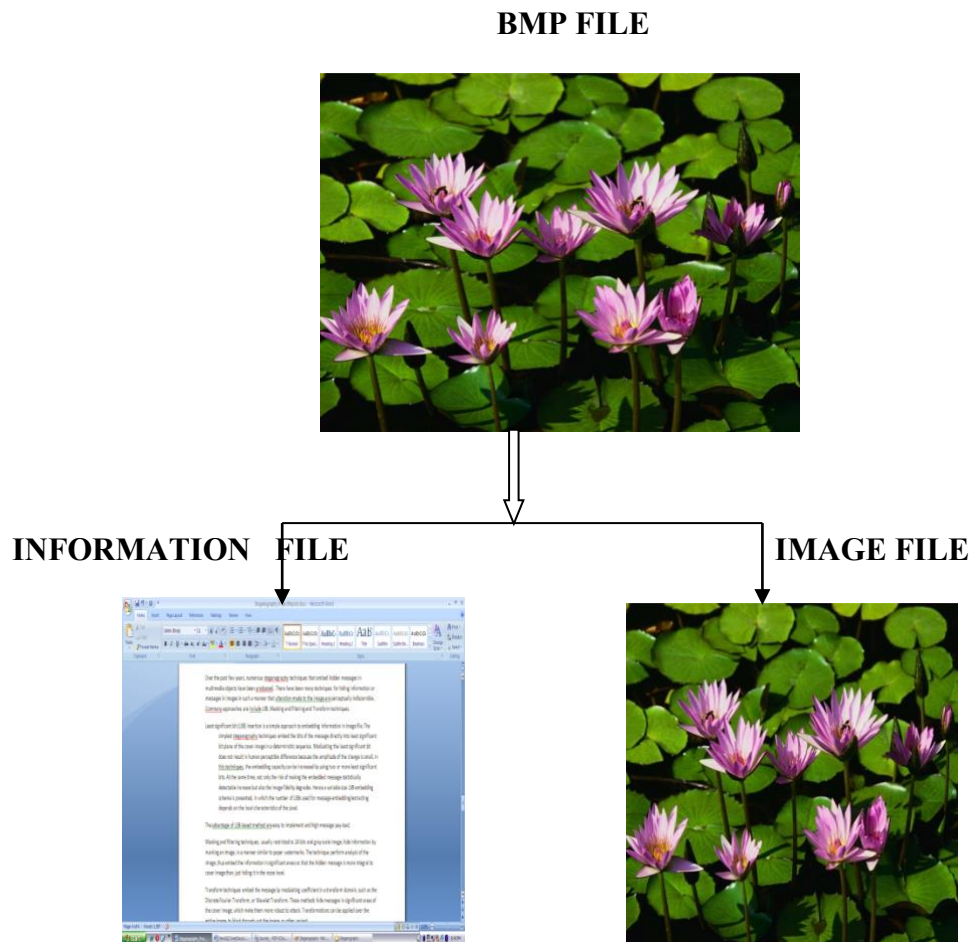


Fig. 5.3 Get the hidden information in an image file by Decryption Process

5.4 Sample Code

```
using System;
using System.Drawing;
using System.Windows.Forms;
using System.IO;

namespace Text2Image
{
    public partial class FrmSteganography : Form
    {
        public FrmSteganography()
        {
            InitializeComponent();

            //public values:
            string loadedTrueImagePath, loadedFilePath, saveToImage, DLoadImagePath, DSaveFilePath;
            int height, width;
            long fileSize, fileNameSize;
            Image loadedTrueImage, DecryptedImage ,AfterEncryption;
            Bitmap loadedTrueBitmap, DecryptedBitmap;
            Rectangle previewImage = new Rectangle(20,160,490,470);
            bool canPaint = false, EncriptionDone = false;
            byte[] fileContainer;

            private void EnImageBrowse_btn_Click(object sender, EventArgs e)
            {
                if (openFileDialog1.ShowDialog() == DialogResult.OK)
                {
                    loadedTrueImagePath = openFileDialog1.FileName;
                    EnImage_tbx.Text = loadedTrueImagePath;
                    loadedTrueImage = Image.FromFile(loadedTrueImagePath);
                    height = loadedTrueImage.Height;
                    width = loadedTrueImage.Width;
                    loadedTrueBitmap = new Bitmap(loadedTrueImage);

                    FileInfo imginf = new FileInfo(loadedTrueImagePath);
                    float fs = (float)imginf.Length / 1024;
                    ImageSize_lbl.Text = smalldecimal(fs.ToString(), 2) + " KB";
                    ImageHeight_lbl.Text = loadedTrueImage.Height.ToString() + " Pixel";
                    ImageWidth_lbl.Text = loadedTrueImage.Width.ToString() + " Pixel";
                    double cansave = (8.0 * ((height * (width / 3) * 3) / 3 - 1)) / 1024;
                    CanSave_lbl.Text = smalldecimal(cansave.ToString(), 2) + " KB";

                    canPaint = true;
                    this.Invalidate();
                }
            }
        }
    }
}
```

```

private string smalldecimal(string inp, int dec)
{
    int i;
    for (i = inp.Length - 1; i > 0; i--)
        if (inp[i] == '.')
            break;
    try
    {
        return inp.Substring(0, i + dec + 1);
    }
    catch
    {
        return inp;
    }
}

private void EnFileBrowse_btn_Click(object sender, EventArgs e)
{
    if (openFileDialog2.ShowDialog() == DialogResult.OK)
    {
        loadedFilePath = openFileDialog2.FileName;
        EnFile_tbx.Text = loadedFilePath;
        FileInfo finfo = new FileInfo(loadedFilePath);
        fileSize = finfo.Length;
        fileNameSize = justFName(loadedFilePath).Length;
    }
}

private void Encrypt_btn_Click(object sender, EventArgs e)
{
    if (saveFileDialog1.ShowDialog() == DialogResult.OK)
    {
        saveToImage = saveFileDialog1.FileName;
    }
    else
        return;
    if (EnImage_tbx.Text == String.Empty || EnFile_tbx.Text == String.Empty)
    {
        MessageBox.Show("Encrypton information is incomplete!\nPlease complete them frist.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    if (8*((height * (width/3)*3)/3 - 1) < fileSize + fileNameSize)
    {
        MessageBox.Show("File size is too large!\nPlease use a larger image to hide this file.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        return;
    }
    fileContainer = File.ReadAllBytes(loadedFilePath);
    EncryptLayer();
}

```

```

private void EncryptLayer()
{
    toolStripStatusLabel1.Text = "Encrypting... Please wait";
    Application.DoEvents();
    long FSize = fileSize;
    Bitmap changedBitmap = EncryptLayer(8, loadedTrueBitmap, 0, (height * (width/3)*3) / 3
- fileNameSize - 1, true);
    FSize -= (height * (width / 3) * 3) / 3 - fileNameSize - 1;
    if(FSize > 0)
    {
        for (int i = 7; i >= 0 && FSize > 0; i--)
        {
            changedBitmap = EncryptLayer(i, changedBitmap, (((8 - i) * height * (width / 3) * 3)
/ 3 - fileNameSize - (8 - i)), (((9 - i) * height * (width / 3) * 3) / 3 - fileNameSize - (9 - i)), false);
            FSize -= (height * (width / 3) * 3) / 3 - 1;
        }
    }
    changedBitmap.Save(saveToImage);
    toolStripStatusLabel1.Text = "Encrypted image has been successfully saved.";
    EncryptionDone = true;
    AfterEncryption = Image.FromFile(saveToImage);
    this.Invalidate();
}

```

```

private Bitmap EncryptLayer(int layer, Bitmap inputBitmap, long startPosition, long
endPosition, bool writeFileName)
{
    Bitmap outputBitmap = inputBitmap;
    layer--;
    int i = 0, j = 0;
    long FNSize = 0;
    bool[] t = new bool[8];
    bool[] rb = new bool[8];
    bool[] gb = new bool[8];
    bool[] bb = new bool[8];
    Color pixel = new Color();
    byte r, g, b;

    if (writeFileName)
    {
        FNSize = fileNameSize;
        string fileName = justFName(loadedFilePath);

        //write fileName:
        for (i = 0; i < height && i * (height / 3) < fileNameSize; i++)
            for (j = 0; j < (width / 3) * 3 && i * (height / 3) + (j / 3) < fileNameSize; j++)
            {

```



```

        byte2bool((byte)fileName[i * (height / 3) + j / 3], ref t);
        pixel = inputBitmap.GetPixel(j, i);
        r = pixel.R;
        g = pixel.G;
        b = pixel.B;
        byte2bool(r, ref rb);
        byte2bool(g, ref gb);
        byte2bool(b, ref bb);
        if (j % 3 == 0)
        {
            rb[7] = t[0];
            gb[7] = t[1];
            bb[7] = t[2];
        }
        else if (j % 3 == 1)
        {
            rb[7] = t[3];
            gb[7] = t[4];
            bb[7] = t[5];
        }
        else
        {
            rb[7] = t[6];
            gb[7] = t[7];
        }
        Color result = Color.FromArgb((int)bool2byte(rb), (int)bool2byte(gb),
(int)bool2byte(bb));
        outputBitmap.SetPixel(j, i, result);
    }
    i--;
}
//write file (after file name):
int tempj = j;

for (; i < height && i * (height / 3) < endPosition - startPosition + FNSize &&
startPosition + i * (height / 3) < fileSize + FNSize; i++)
    for (j = 0; j < (width / 3) * 3 && i * (height / 3) + (j / 3) < endPosition - startPosition +
FNSize && startPosition + i * (height / 3) + (j / 3) < fileSize + FNSize; j++)
    {
        if (tempj != 0)
        {
            j = tempj;
            tempj = 0;
        }
        byte2bool((byte)fileContainer[startPosition + i * (height / 3) + j / 3 - FNSize], ref t);
        pixel = inputBitmap.GetPixel(j, i);
        r = pixel.R;
        g = pixel.G;
        b = pixel.B;
        byte2bool(r, ref rb);
    }

```



```

        byte2bool(g, ref gb);
        byte2bool(b, ref bb);
        if (j % 3 == 0)
        {
            rb[layer] = t[0];
            gb[layer] = t[1];
            bb[layer] = t[2];
        }
        else if (j % 3 == 1)
        {
            rb[layer] = t[3];
            gb[layer] = t[4];
            bb[layer] = t[5];
        }
        else
        {
            rb[layer] = t[6];
            gb[layer] = t[7];
        }
        Color result = Color.FromArgb((int)bool2byte(rb), (int)bool2byte(gb),
(int)bool2byte(bb));
        outputBitmap.SetPixel(j, i, result);

    }
    long tempFS = fileSize, tempFNS = fileNameSize;
    r = (byte)(tempFS % 100);
    tempFS /= 100;
    g = (byte)(tempFS % 100);
    tempFS /= 100;
    b = (byte)(tempFS % 100);
    Color flenColor = Color.FromArgb(r,g,b);
    outputBitmap.SetPixel(width - 1, height - 1, flenColor);

    r = (byte)(tempFNS % 100);
    tempFNS /= 100;
    g = (byte)(tempFNS % 100);
    tempFNS /= 100;
    b = (byte)(tempFNS % 100);
    Color fnlenColor = Color.FromArgb(r,g,b);
    outputBitmap.SetPixel(width - 2, height - 1, fnlenColor);

    return outputBitmap;
}

private void DecryptLayer()
{
    toolStripStatusLabel1.Text = "Decrypting... Please wait";
    Application.DoEvents();
    int i, j = 0;

```

```

bool[] t = new bool[8];
bool[] rb = new bool[8];
bool[] gb = new bool[8];
bool[] bb = new bool[8];
Color pixel = new Color();
byte r, g, b;
pixel = DecryptedBitmap.GetPixel(width - 1, height - 1);
long fSize = pixel.R + pixel.G * 100 + pixel.B * 10000;
pixel = DecryptedBitmap.GetPixel(width - 2, height - 1);
long fNameSize = pixel.R + pixel.G * 100 + pixel.B * 10000;
byte[] res = new byte[fSize];
string resFName = "";
byte temp;

//Read file name:
for (i = 0; i < height && i * (height / 3) < fNameSize; i++)
    for (j = 0; j < (width / 3) * 3 && i * (height / 3) + (j / 3) < fNameSize; j++)
    {
        pixel = DecryptedBitmap.GetPixel(j, i);
        r = pixel.R;
        g = pixel.G;
        b = pixel.B;
        byte2bool(r, ref rb);
        byte2bool(g, ref gb);
        byte2bool(b, ref bb);
        if (j % 3 == 0)
        {
            t[0] = rb[7];
            t[1] = gb[7];
            t[2] = bb[7];
        }
        else if (j % 3 == 1)
        {
            t[3] = rb[7];
            t[4] = gb[7];
            t[5] = bb[7];
        }
        else
        {
            t[6] = rb[7];
            t[7] = gb[7];
            temp = bool2byte(t);
            resFName += (char)temp;
        }
    }

//Read file on layer 8 (after file name):
int tempj = j;
i--;

```

```

for (; i < height && i * (height / 3) < fSize + fNameSize; i++)
    for (j = 0; j < (width / 3) * 3 && i * (height / 3) + (j / 3) < (height * (width / 3) * 3) / 3 -
1 && i * (height / 3) + (j / 3) < fSize + fNameSize; j++)
    {
        if (tempj != 0)
        {
            j = tempj;
            tempj = 0;
        }
        pixel = DecryptedBitmap.GetPixel(j, i);
        r = pixel.R;
        g = pixel.G;
        b = pixel.B;
        byte2bool(r, ref rb);
        byte2bool(g, ref gb);
        byte2bool(b, ref bb);
        if (j % 3 == 0)
        {
            t[0] = rb[7];
            t[1] = gb[7];
            t[2] = bb[7];
        }
        else if (j % 3 == 1)
        {
            t[3] = rb[7];
            t[4] = gb[7];
            t[5] = bb[7];
        }
        else
        {
            t[6] = rb[7];
            t[7] = gb[7];
            temp = bool2byte(t);
            res[i * (height / 3) + j / 3 - fNameSize] = temp;
        }
    }

```

//Read file on other layers:

```
long readedOnL8 = (height * (width/3)*3) / 3 - fNameSize - 1;
```

```

for (int layer = 6; layer >= 0 && readedOnL8 + (6 - layer) * ((height * (width / 3) * 3) / 3 -
1) < fSize; layer--)
    for (i = 0; i < height && i * (height / 3) + readedOnL8 + (6 - layer) * ((height * (width /
3) * 3) / 3 - 1) < fSize; i++)
        for (j = 0; j < (width / 3) * 3 && i * (height / 3) + (j / 3) + readedOnL8 + (6 - layer) *
((height * (width / 3) * 3) / 3 - 1) < fSize; j++)
        {
            pixel = DecryptedBitmap.GetPixel(j, i);
            r = pixel.R;
            g = pixel.G;

```

```

        b = pixel.B;
        byte2bool(r, ref rb);
        byte2bool(g, ref gb);
        byte2bool(b, ref bb);
        if (j % 3 == 0)
        {
            t[0] = rb[layer];
            t[1] = gb[layer];
            t[2] = bb[layer];
        }
        else if (j % 3 == 1)
        {
            t[3] = rb[layer];
            t[4] = gb[layer];
            t[5] = bb[layer];
        }
        else
        {
            t[6] = rb[layer];
            t[7] = gb[layer];
            temp = bool2byte(t);
            res[i * (height / 3) + j / 3 + (6 - layer) * ((height * (width / 3) * 3) / 3 - 1) +
readedOnL8] = temp;
        }
    }
}

if (File.Exists(DSaveFilePath + "\\" + resFName))
{
    MessageBox.Show("File \"" + resFName + "\" already exist please choose another path
to save file", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    return;
}
else
    File.WriteAllBytes(DSaveFilePath + "\\" + resFName, res);
toolStripStatusLabel1.Text = "Decrypted file has been successfully saved.";
Application.DoEvents();
}

private void byte2bool(byte inp, ref bool[] outp)
{
    if(inp >= 0 && inp <= 255)
        for (short i = 7; i >= 0; i--)
        {
            if (inp % 2 == 1)
                outp[i] = true;
            else
                outp[i] = false;
            inp /= 2;
        }
    else

```

```

        throw new Exception("Input number is illegal.");
    }

    private byte bool2byte(bool[] inp)
    {
        byte outp = 0;
        for (short i = 7; i >= 0; i--)
        {
            if (inp[i])
                outp += (byte)Math.Pow(2.0, (double)(7-i));
        }
        return outp;
    }

    private void Decrypt_btn_Click(object sender, EventArgs e)
    {

        if (DeSaveFile_tbx.Text == String.Empty || DeLoadImage_tbx.Text == String.Empty)
        {
            MessageBox.Show("Text boxes must not be empty!", "Error", MessageBoxButtons.OK,
            MessageBoxIcon.Error);

            return;
        }

        if (System.IO.File.Exists(DeLoadImage_tbx.Text) == false)
        {
            MessageBox.Show("Select image file.", "Error", MessageBoxButtons.OK,
            MessageBoxIcon.Exclamation);
            DeLoadImage_tbx.Focus();
            return;
        }

        DecryptLayer();
    }

    private void DeLoadImageBrowse_btn_Click(object sender, EventArgs e)
    {
        if (openFileDialog3.ShowDialog() == DialogResult.OK)
        {
            DLoadImagePath = openFileDialog3.FileName;
            DeLoadImage_tbx.Text = DLoadImagePath;
            DecryptedImage = Image.FromFile(DLoadImagePath);
            height = DecryptedImage.Height;
            width = DecryptedImage.Width;
            DecryptedBitmap = new Bitmap(DecryptedImage);
        }
    }

```

```

        FileInfo imginf = new FileInfo(DLoadImagePath);
        float fs = (float)imginf.Length / 1024;
        ImageSize_lbl.Text = smalldecimal(fs.ToString(), 2) + " KB";
        ImageHeight_lbl.Text = DecryptedImage.Height.ToString() + " Pixel";
        ImageWidth_lbl.Text = DecryptedImage.Width.ToString() + " Pixel";
        double cansave = (8.0 * ((height * (width / 3) * 3) / 3 - 1)) / 1024;
        CanSave_lbl.Text = smalldecimal(cansave.ToString(), 2) + " KB";

        canPaint = true;
        this.Invalidate();
    }
}

private void DeSaveFileBrowse_btn_Click(object sender, EventArgs e)
{
    if (folderBrowserDialog1.ShowDialog() == DialogResult.OK)
    {
        DSaveFilePath = folderBrowserDialog1.SelectedPath;
        DeSaveFile_tbx.Text = DSaveFilePath;
    }
}

private void Form1_Paint(object sender, PaintEventArgs e)
{
    if(canPaint)
    try
    {
        if (!EncrptionDone)
            e.Graphics.DrawImage(loadedTrueImage, previewImage);
        else
            e.Graphics.DrawImage(AfterEncryption, previewImage);
    }
    catch
    {
        e.Graphics.DrawImage(DecryptedImage, previewImage);
    }
}

private string justFName(string path)
{
    string output;
    int i;
    if (path.Length == 3) // i.e: "C:\\"
        return path.Substring(0, 1);
    for (i = path.Length - 1; i > 0; i--)
        if (path[i] == '\\')
            break;
    output = path.Substring(i + 1);
    return output;
}

```

```

private string justEx(string fName)
{
    string output;
    int i;
    for (i = fName.Length - 1; i > 0; i--)
        if (fName[i] == '.')
            break;
    output = fName.Substring(i + 1);
    return output;
}

private void Close_btn_Click(object sender, EventArgs e)
{
    this.Close();
}

private void linkLabel1_LinkClicked(object sender, LinkLabelLinkClickedEventArgs e)
{
    System.Diagnostics.Process.Start("http://www.programmer2programmer.net");
}
}
}

```

Conclusion

Steganography is a really interesting subject and outside of the mainstream cryptography and system administration that most of us deal with day after day.

Steganography can be used for hidden communication. We have explored the limits of steganography theory and practice. We printed out the enhancement of the image steganography system using LSB approach to provide a means of secure communication. A stego-key has been applied to the system during embedment of the message into the cover image.

This steganography application software provided for the purpose to how to use any type of image formats to hiding any type of files inside them. The master work of this application is in supporting any type of pictures without need to convert to bitmap, and lower limitation on file size to hide, because of using maximum memory space in pictures to hide the file.

Since ancient times, man has found a desire in the ability to communicate covertly. The recent explosion of research in watermarking to protect intellectual property is evidence that steganography is not just limited to military or espionage applications. Steganography, like cryptography, will play an increasing role in the future of secure communication in the “digital world”.

Future scope

This Project can be easily implemented under various situations. We can add new features as and when we require. Re usability is possible as and when require in this project. There is flexibility in all modules.

There are some limitations for the current system to which solutions can be provided as a future development.

The system is not configured for multimedia conversion for output file at this time. The concept of Multimedia file transformation can be used to achieved this.

The software is not accessible to every operating system. It can be deployed on an emulator so that everybody who us connected to emulator can use it.

Software installation is not done. Portable third party software is used as sort of pre-installed system installation files as required.

As future developments the following can be done:

The application used as os preinstalled system software after which there is no need to grant permission to software for every first attempts.

Multiple formats can be supported fir output file but may risk to a security issues as there is possibility of easily access of files.

We can add multiple themes to software for those user who to customize their experience.

Bibliography

Websites

Following websites are referring to create this project reports.

- <http://www.google.com>
- <http://www.microsoft.com>
- <http://www.programmer2programmer.net>
- <http://www.codeproject.com>
- <http://www.asp.net>
- <http://www.asp123.com>
- <http://www.wikipedia.org>

Books

Following books and eBooks are used to complete this project reports.

- Mastering C# (Paperback)
- SQL Server Bible (Paperback)
- .NET Black Book (Paperback)
- Professional C#, 2nd Edition (Paperback)
- Professional ASP.NET (Paperback)
- MCAD/MCSD Self-Paced Training Kit: Developing Web Applications with Microsoft® Visual Basic® .NET and Microsoft Visual C#® .NET, Second Edition
- MCAD/MCSE/MCDBA Self-Paced Training Kit: Microsoft SQL Server 2000 Database Design and Implementation, Exam 70-229, Second Edition

APPENDIX

9.1 Introduction to Operational Steps and Methods

User needs to run the application. The user has two tab options – encrypt and decrypt. If user select encrypt, application give the screen to select image file, information file and option to save the image file. If user select decrypt, application gives the screen to select only image file and ask path where user want to save the secrete file.

This project has two methods – Encrypt and Decrypt.

In encryption the secrete information is hiding in with any type of image file.

Decryption is getting the secrete information from image file.

9.2 Introduction to Theories used to Adapt Particular Design

Steganography system requires any type of image file and the information or message that is to be hidden. It has two modules encrypt and decrypt. Microsoft .Net framework prepares a huge amount of tool and options for programmers that they simples programming. One of .Net tools for pictures and images is auto-converting most types of pictures to BMP format. I used this tool in this software called “Steganography” that is written in C#.Net language and you can use this software to hide your information in any type of pictures without any converting its format to BMP (software converts inside it).

The algorithm used for Encryption and Decryption in this application provides using several layers' lieu of using only LSB layer of image. Writing data starts from last layer (8st or LSB layer); because significant of this layer is least and every upper layer has doubled significant from its down layer. So every step we go to upper layer image quality decreases and image retouching transpires.